AGROCARE – A LOW-CODE SMART AGRICULTURE MANAGEMENT SYSTEM USING ZOHO CREATOR

A PROJECT REPORT

Submitted by

UMASANKAR N

(2116240701574)

in partial fulfilment of the End Semester Examination for the course

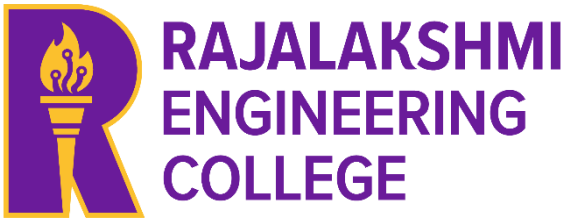
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "AGROCARE – A LOW-CODE SMART AGRICULTURE MANAGEMENT SYSTEM USING ZOHO CREATOR" is the bonafide work of Umasankar N (2116240701574), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr.MANORANJINI J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr.BHUVANESWARAN B
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Umasankar N (2116240701574), hereby declare that the project report titled "AGROCARE – A LOW-CODE SMART AGRICULTURE MANAGEMENT SYSTEM USING ZOHO CREATOR", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT

Umasankar N

2116240701574

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mrs. Kaviya S, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

UMASANKAR N

ABSTRACT

Small and marginal farmers in India usually choose crops on the basis of habit rather than a match between soil type, season, water availability and soil pH. Pest and disease problems are noticed late, advice from agricultural officers reaches farmers slowly through phone calls or visits, and requests for seeds and fertilizers are handled on paper with no live view of stock. Records of irrigation, fertilizer use and crop health are rarely kept, so neither the farmer nor the department can see problems building up in a village.

AgroCare is a smart agriculture management application built on Zoho Creator to address these problems. It contains 21 forms linked through lookups and four subforms, covering farmers, farm plots, crops, seeds, fertilizers, crop plans, health, irrigation and fertilizer logs, pest and disease reports, officer advice, inventory, farmer requests and alerts. Deluge scripts and 15 form workflows plus one daily schedule automate the work: a crop suitability score out of 100 (soil 35, season 30, water 20, pH 15), automatic officer assignment, request costing and stock deduction on approval, low-stock alerts, a crop health score, irrigation and fertilizer reminders, a community early-warning rule that raises a high-risk alert when three or more farmers in a village report the same disease, and weather-based advisories. Notifications go out through email and Zoho Cliq, which is connected through a Creator connection and the invokeUrl task. Twelve reports (list, Kanban, pivot chart and pivot table) feed three role-based dashboards for the farmer, the agricultural officer and the administrator, with access controlled through roles and permissions.

The application was tested form by form and workflow by workflow using sample data. Together the modules replace paper registers and phone follow-up with a single system in which a reported problem is assigned, advised and tracked to resolution, and stock is reduced automatically when a request is approved. The project shows that a complete, automated, multi-role agricultural application can be built with low-code tools in a fraction of the time that custom development would need.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Smart Agriculture; Crop Recommendation; Role-Based Access Control*

PROJECT AT A GLANCE

Attribute	Details
Project Title	AgroCare – Smart Agriculture Management System
Application Domain	Agriculture (crop advisory, crop monitoring, and seed and fertilizer distribution)
Student Name & Register No.	Umasankar N – 2116240701574
Zoho Creator Edition	Trial edition
Application Name (Link Name)	AgroCare (link name: agrocare)
Published / Live URL	https://creatorapp.zoho.in/umasankar.n.2024.cse_raja280/agrocare
Demo Credentials (per role)	Owner account (Umasankar N) only; no separate demo logins were created
Number of Forms / Subforms	21 / 4
Number of Reports (by type)	List: 3 | Kanban: 2 | Calendar: 0 | Pivot chart: 6 | Pivot table: 1 (total 12)
Number of Pages / Dashboards	3 (Farmer_Dashboard, Officer_Dashboard, Admin_Dashboard)
Number of Workflows	Form: 15 | Schedule: 1 | Approval: 0 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	13 scripts inside workflows and pages (DS-01 to DS-13); no stand-alone functions or custom actions
Integrations	Zoho Cliq (connection + invokeUrl), Email (sendmail)
Roles & Profiles Defined	CEO (owner/administrator), Agricultural Officer, Farmer
Environments Used	Development only (trial account has a single environment)
Date of Production Deployment	Not published to production; tested on 10 November 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 14
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 17
4.6 Access Control Design 18
IMPLEMENTATION 20
5.1 Application Setup 20
5.2 Forms and Data Capture 20

5.3 Reports 21
5.4 Pages and Dashboards 22
5.5 Deluge Scripting 23
5.6 Workflows and Automation 24
5.7 Integrations 25
5.8 User Management and Role-Based Access 26
5.9 AI and Analytics Features 27
5.10 Pro-Code Extension 28
5.11 Mobile and Portal Access 28
TESTING AND DEBUGGING 29
6.1 Testing Strategy 29
6.2 Test Cases 29
6.3 Debugging Deluge Scripts 34
6.4 Defect Log 35
6.5 User Acceptance Testing 36
DEPLOYMENT, SECURITY AND GOVERNANCE 37
7.1 Environments and Deployment Strategy 37
7.2 Performance Optimisation 38
7.3 Security Features 38
7.4 Governance and Administration 39
RESULTS AND DISCUSSION 40
8.1 Application Output 40
8.2 Achievement of Objectives 42
8.3 Comparison with the Existing System 42
8.4 Limitations of the Low-Code Approach 42
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 43
CONCLUSION AND FUTURE ENHANCEMENTS 44
9.1 Conclusion 44
9.2 Future Enhancements 44
9.3 Learning Outcomes and Career Relevance 44
REFERENCES 45
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 46
APPENDIX B – COURSE OUTCOME MAPPING 48
APPENDIX C – COMPLETE DELUGE SOURCE CODE 49
APPENDIX D – USER MANUAL 57
D.1 Accessing the Application 57
D.2 Instructions for the Farmer 57

D.3 Instructions for the Agricultural Officer 57
D.4 Instructions for the Administrator..... 57
APPENDIX E – EVALUATION RUBRIC 59
APPENDIX F – VIVA-VOCE RECORD SHEET 61
APPENDIX F – VIVA-VOCE RECORD SHEET 61

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 9
Table 3.5: Use Case Description – Report Pest or Disease 11
Table 3.6: Use Case Description – Approve Request and Deduct Stock 11
Table 4.1: Entities and Relationships 13
Table 4.2: Forms of the Application by Module 14
Table 4.3: Field Specification – Farm_Plot Form 14
Table 4.4: Field Specification – Pest_Disease_Report Form 15
Table 4.5: Role–Permission Matrix 19
Table 5.1: Reports Implemented 21
Table 5.2: Deluge Scripts Implemented 23
Table 5.3: Workflows Implemented 24
Table 5.4: Integrations Implemented 26
Table 6.1: Test Cases 29
Table 6.2: Defect Log 35
Table 6.3: User Acceptance Feedback 36
Table 7.1: Deployment Checklist 37
Table 7.2: Performance Optimisation Results 38
Table 7.3: Security Controls Implemented 38
Table 8.1: Mapping of Objectives to Outcomes 42
Table 8.2: Existing System versus Proposed Application 42
Table 8.3: Placement of AgroCare Features on the Development Spectrum 43
Table A.1: Feature Coverage Checklist 46
Table B.1: Mapping of Course Outcomes to Report Chapters 48
Table E.1: Evaluation Rubric for the Individual Project 59
Table E.2: Performance Level Descriptors 60
Table F.1: Viva-Voce Questions and Responses 61
Table F.2: Marks Summary 61
Table F.2: Marks Summary 61

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 10
Figure 4.1: System Architecture of the Application..... 12
Figure 4.2: Entity–Relationship Diagram of the Application 13
Figure 4.3: Workflow / Process Flow for Pest and Disease Case Handling..... 16
Figure 4.4: Workflow / Process Flow for Request Approval and Stock Deduction .. 17
Figure 4.5: Navigation Structure of the Application by Role 18
Figure 4.6: Wireframe of the Admin_Dashboard Page 18
Figure 5.1: Application Left-Menu Navigation and the Pest Disease Report Form .. 20
Figure 5.2: Crop_Plan Form in the Form Builder 21
Figure 5.3: Farmer_Request Form in Live Mode 21
Figure 5.4: Case_Board (Kanban) Report with Case 6 in Advisory Given..... 22
Figure 5.5: Admin_Dashboard Page with KPI Cards 23
Figure 5.6: Pest Report Setup Workflow on Pest_Disease_Report (enabled)..... 25
Figure 5.7: Workflow Area: Form Workflows, Schedules and Functions (Crop PH Check enabled)..... 26
Figure 5.8: Roles Hierarchy: CEO, Agricultural Officer and Farmer 27
Figure 5.9: Ranked Crop Recommendation Output (All Crop Recommendations report) 28
Figure 6.1: TC-01: Duplicate Phone Number Rejected on the Farmer Form..... 33
Figure 6.2: TC-10: Request 4 Approved, Stock Deducted = Yes 33
Figure 6.3: TC-05: Fertilizer Usage Record with Total Cost 13500..... 33
Figure 6.4: TC-16: Weather Updates Report (Alerts Created = 0) 34
Figure 6.5: Validation Message from the pH Range Check Script (Min pH cannot be greater than Max pH)..... 34
Figure 7.1: Application Settings: URL, Permissions and Developer Tools 37
Figure 7.2: Schema Builder: Relationships Between the Application Forms 39
Figure 8.1: Scenario Step 1 – Crop Recommendation with Suitability Score..... 40
Figure 8.2: Scenario Step 2 – Pest Report 6 Submitted: Farmer Filled, Officer Ravi Assigned, Under Review 41
Figure 8.3: Scenario Step 3 – Officer Advice Saved; Case 6 Moves to Advisory Given..... 41
Figure 8.4: Scenario Step 4 – Request Approved and Stock Deducted 41
Figure 8.4: Scenario Step 4 – Request Approved and Stock Deducted 41

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CO	Course Outcome
DS	Deluge Script (script identifier)
ER	Entity–Relationship
FR / NFR	Functional / Non-Functional Requirement
HTML	HyperText Markup Language
IEEE	Institute of Electrical and Electronics Engineers
KPI	Key Performance Indicator
OAuth	Open Authorization
pH	Potential of Hydrogen (soil acidity scale)
RBAC	Role-Based Access Control
REST	Representational State Transfer
TC	Test Case
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Agriculture is the main livelihood of a large part of the population in Tamil Nadu and the rest of India, and most farms are small holdings of one to five acres. A farmer's income depends on a chain of decisions: which crop to grow in a given season, how much to irrigate, which fertilizer to apply and when, and how quickly a pest or disease is recognised and treated. These decisions depend on the soil type, soil pH, water availability, season and weather of each plot.

The process followed today is largely manual. Farmers rely on experience and neighbours for crop choice, buy seeds and fertilizers from local shops or ask the agriculture department for them, and approach the agricultural officer by phone or in person when a crop problem appears. Officers keep paper registers or spreadsheets of the cases they handle and have no consolidated view of how many farmers in a village are facing the same disease. The department's store manages stock of seeds and fertilizers in registers.

AgroCare is designed for a realistic setting of a block-level agriculture office that serves a group of villages. Its users are farmers, agricultural officers who advise them, and an administrator who manages the catalogue of crops, seeds and fertilizers, the stock and the requests.

1.2 Problem Statement

Farmers do not have one place to get crop advice, record crop activity, report pest and disease problems, receive timely warnings and request inputs, while officers and administrators lack real-time visibility of cases, risk and stock. The result is poor crop selection, late treatment of diseases and inefficient distribution of seeds and fertilizers. The specific pain points are listed below.

- Crop selection is based on habit, with no structured match against soil, season, water and pH.
- Pest and disease reports reach officers late and are tracked on paper, so the status of a case is unknown to the farmer.

- Outbreaks that affect many farmers in one village are noticed only after they spread.
- Irrigation and fertilizer schedules are forgotten because no reminder system exists.
- Seed and fertilizer stock and farmer requests are managed manually, causing stock-outs and delays.
- There are no role-based dashboards or reports for decision making.

1.3 Objectives

- To design a normalised data model and forms for farmers, plots, crops, crop plans, monitoring logs, reports, inventory, requests and alerts.
- To automate crop scoring, officer assignment, request costing, stock deduction, health scoring, reminders and disease-risk warnings using Deluge scripting and workflows.
- To provide role-based dashboards and reports for farmers, agricultural officers and administrators.
- To integrate the application with Zoho Cliq and email so that officers and farmers are notified in real time.
- To control access with roles and permissions so each user sees only the data and screens relevant to the role.
- To test the application with sample data and prepare it for deployment with governance controls.

1.4 Scope of the Project

In scope: farmer and farm-plot registration; crop, soil, seed, fertilizer and disease master data; crop suitability recommendation with a score; crop plans with sowing and expected harvest dates; crop health, irrigation and fertilizer logging; pest and disease reporting with officer advice and case-status tracking; community early warning and disease-risk level; weather-based advisory using manually entered weather data; irrigation, fertilizer and low-stock alerts; seed and fertilizer inventory and requests with automatic stock deduction; crop guides and farming articles; reports, dashboards, notifications and role-based access. The application is accessed

through the web and the Zoho Creator mobile app. The three user roles are Administrator, Agricultural Officer and Farmer.

Out of scope: a live weather feed from an external API (the design allows it through invokeUrl, but weather is entered manually in this version), automatic disease detection from photographs, online payment, SMS delivery and a public farmer portal. These are discussed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

A low-code platform is suitable because AgroCare is a data-centred, workflow-heavy business application whose requirements will keep changing with seasons and department policy. Zoho Creator provides forms, relationships, reports, dashboards, role-based security, hosting and a mobile app out of the box, and Deluge allows the remaining business rules to be written in a few lines [1], [2]. This reduced development time, allowed the application to be changed quickly when the feature list was extended, and means that a department employee with basic training can maintain it without a development team. Hosting, backup and security are handled by the platform, which keeps the cost low for a government or non-profit user. The limitations of this approach are discussed in Section 8.4.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing system a farmer decides the crop on his own or on the advice of neighbours. When a pest or disease problem appears, he phones or visits the agricultural officer, who notes the case in a register and advises verbally. Seeds and fertilizers are requested by a written application to the department store, where the store keeper checks the stock register by hand and issues the items. Weather information is taken from television or radio, and no one links it to disease risk for a particular crop. Irrigation and fertilizer schedules depend on the farmer's memory. Figure 2.1 shows this flow.

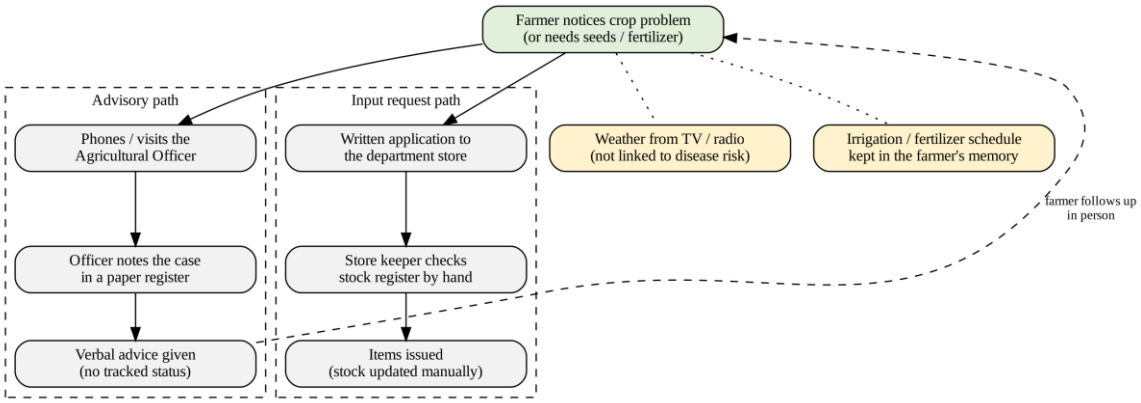


Figure 2.1: As-Is Process Flow of the Existing System

Every hand-over in this flow depends on a person remembering to pass the information on, and the farmer has no way to see the progress of a case or a request except by following up in person.

2.2 Limitations of the Existing System

- No structured crop recommendation; choices are not matched to soil, season, water and pH.
- Cases are tracked on paper, so status and history are not visible to the farmer or the department.
- Village-level outbreaks are not detected early.
- No reminders for irrigation, fertilizer or low stock.
- Stock and requests are handled manually, leading to errors and delays.

- No dashboards or reports, so planning depends on guesswork.
- No access control; paper records can be altered or lost.

2.3 Low-Code versus Traditional Development

Low-code platforms let applications be assembled mainly through visual modelling of data, user interfaces and processes, with code used only where the platform's building blocks are not enough [3]. Industry analysts track enterprise low-code application platforms as a distinct market, and Zoho is among the vendors evaluated [4]. Table 2.1 compares the two approaches for AgroCare. The development-time figures are estimates of effort, not measurements.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	About 10–12 weeks (database design, back end, front end, testing)	About 3–4 weeks of part-time work
Skills required	Back-end, front-end, database and DevOps skills	Data modelling, form design and basic Deluge scripting
Hosting & infrastructure	Server, database and security set up and paid for separately	Provided by the platform
Mobile application	Separate Android/iOS development	Zoho Creator mobile app available immediately
Security & access control	Authentication and permissions coded manually	Roles, permissions and record-level criteria configured
Maintenance & change	Code changes, rebuild and redeploy	Edit a form or script and publish
Customisation limits	Unlimited	Limited by platform features and Deluge limits
Cost	High development and hosting cost	Subscription only; free or education plan for the project

2.4 Literature Survey

Table 2.2 summarises the sources studied. They include a book and a research article on low-code development [1], [3], an industry analysis of low-code platforms [4], a research paper on crop recommendation for precision agriculture [5], and the official Zoho documentation used while building the application [2], [6].

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam and O. Kalnichevski [1]	Low-Code Development: A Practical Guide	Explains low-code concepts, citizen development and platform capabilities	Foundation for choosing a low-code approach
2	A. C. Bock and U. Frank, 2021 [3]	Low-Code Platform	Describes what a low-code platform is, its typical components and the promises and limits of the approach	Definition of low-code used in Chapters 1 and 2; supports the limitations in Section 8.4
3	Gartner, Inc., 2025 [4]	Magic Quadrant for Enterprise Low-Code Application Platforms	Analyses growth and adoption of low-code platforms and evaluates vendors, including Zoho	Supports the low-code justification
4	S. Pudumalar et al., 2017 [5]	Crop recommendation system for precision agriculture	Recommends a crop from site-specific soil parameters so that farmers avoid a wrong choice of crop	Shows the need for decision support for farmers; AgroCare uses a transparent rule-based score instead of a trained model
5	Zoho Corporation [2]	Zoho Creator Help Documentation	Describes forms, reports, pages, workflows, connections and permissions	Primary reference for the implementation
6	Zoho Corporation [6]	Deluge Script Reference Guide	Syntax, fetch, insert, sendmail and invokeUrl tasks	Basis for all automation scripts

2.5 Emerging Trends in Low-Code Development

Citizen development allows domain experts such as agriculture staff to build and maintain applications themselves. Hyper-automation combines workflows, scheduled jobs and notifications so that routine tasks run without human action, which is the approach taken in AgroCare for reminders and early warnings. AI-assisted building and AI fields such as Zia are making applications smarter, and composable applications connect small services through APIs and connections. AgroCare uses rule-based scoring as a first step towards these trends, and its design keeps room for live weather APIs and AI-based photo analysis.

2.6 Proposed System

AgroCare replaces the manual process with a single Zoho Creator application. It recommends and ranks crops for each plot, records crop activity, lets farmers report

problems that are automatically assigned to an officer, tracks each case from Submitted to Resolved, warns farmers about disease risk and due activities, detects village-level outbreaks, and manages seed and fertilizer stock and requests with automatic deduction. Dashboards and reports give each role the information it needs, and role-based access keeps data separate.

Each limitation in Section 2.2 is addressed as follows: the crop suitability score replaces habit-based crop choice; online case reports with tracked status replace paper registers; the community early-warning rule detects village-level outbreaks; the daily schedule sends irrigation, fertilizer and low-stock reminders; request costing and automatic stock deduction replace manual stock handling; role-based dashboards provide the missing summary; and roles and permissions protect the records.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Department or application owner	Manage master data (soil, crops, seeds, fertilizers, diseases), inventory, requests, users and reports; view all dashboards
Agricultural Officer	Field officer who advises farmers	Review pest and disease reports, give advice, update case status, monitor high-risk alerts, publish guides and articles
Farmer	End user who owns farm plots	Maintain plots and crop plans, record crop health, irrigation and fertilizer use, report problems, request inputs, read alerts

3.2 Functional Requirements

Each functional requirement below is testable and is referred to again by its FR ID in the test cases of Chapter 6.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow registration of farmers and farm plots with soil type, pH, water availability and location	Admin, Farmer	High
FR-02	The system shall maintain master data for soil types, crops, seeds, fertilizers and diseases	Admin	High
FR-03	The system shall recommend crops for a plot and season and rank them with a score out of 100	Farmer	High
FR-04	The system shall record crop plans and automatically build the plan name and expected harvest date	Farmer	High
FR-05	The system shall record crop health, irrigation and fertilizer use, and calculate costs automatically	Farmer	High
FR-06	The system shall calculate a crop health score and rating for each active crop plan	System	Medium
FR-07	The system shall accept pest and disease reports and automatically assign an officer and set the status	Farmer, System	High
FR-08	The system shall let officers give advice, update the case status and notify the farmer	Officer	High
FR-09	The system shall detect three or more similar reports in a village and raise a high-risk alert	System	High
FR-10	The system shall create disease-risk and weather alerts from entered weather data	Admin, Officer, System	Medium

FR ID	Requirement	Role(s)	Priority
FR-11	The system shall remind farmers when irrigation or fertilizer application is due	System	Medium
FR-12	The system shall let farmers request seeds, fertilizers or advice and calculate the request total	Farmer	High
FR-13	The system shall deduct stock when a request is approved and mark it Out of Stock if stock is short	Admin	High
FR-14	The system shall alert the administrator when stock reaches the reorder level	System	Medium
FR-15	The system shall provide dashboards and reports for each role	All	Medium
FR-16	The system shall notify officers in Zoho Cliq when a new case is reported	System	Medium
FR-17	The system shall restrict access to forms, reports and pages by role	Admin	High
FR-18	The system shall publish crop guides and farming articles	Admin, Officer	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with a few hundred records shall load within about 3 seconds
NFR-02	Security	Users shall access only the forms, reports and pages permitted by their role
NFR-03	Usability	The application shall be usable on the web and the Zoho Creator mobile app with simple, labelled forms
NFR-04	Availability	The application shall be available whenever the Zoho Creator platform is available
NFR-05	Maintainability	Rules shall be kept in separate named workflows so that one rule can be changed without affecting others
NFR-06	Auditability	Record changes shall be traceable through the platform audit trail

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial edition)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)

Category	Specification
Other Zoho services	Zoho Cliq, Zoho Mail (email notifications)
Mobile	Zoho Creator mobile app (Android / iOS)
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases of the system. The Farmer registers plots, obtains crop recommendations, records crop activity, reports problems, requests inputs and reads alerts. The Agricultural Officer gives advice, updates case status, enters weather updates and publishes guides. The Administrator maintains master data and inventory, approves requests and manages users. The System actor represents the workflows and the daily schedule that assign officers, send notifications and run the daily checks.



Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Report Pest or Disease

Item	Description
Use Case ID	UC-01
Use Case Name	Report Pest or Disease
Actor(s)	Farmer; System; Agricultural Officer
Pre-condition	Farmer is logged in and has an active crop plan; at least one officer exists
Main Flow	1. Farmer opens the Pest_Disease_Report form. 2. Selects the crop plan, category, symptoms and severity, and submits. 3. System copies the farmer, assigns an officer by village, and sets the status to Under Review. 4. System emails the officer and posts a message to the Cliq channel.
Alternate Flow	If no officer matches the village, the first available officer is assigned. If no officer exists, the report is saved without assignment.
Post-condition	The case is visible on the officer dashboard and case board with status Under Review
Related FR	FR-07, FR-16

Table 3.6: Use Case Description – Approve Request and Deduct Stock

Item	Description
Use Case ID	UC-02
Use Case Name	Approve Request and Deduct Stock
Actor(s)	Administrator
Pre-condition	A request with items exists with status Pending; inventory records exist
Main Flow	1. Administrator opens the request and sets Status to Approved. 2. System checks stock for every item. 3. If stock is sufficient, quantities are deducted and the request is marked as deducted.
Alternate Flow	If stock is insufficient for any item, the status becomes Out of Stock and nothing is deducted.
Post-condition	Inventory reflects the new stock; a low-stock alert is raised at the next daily run if the reorder level is reached
Related FR	FR-13, FR-14

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the layered architecture of AgroCare.

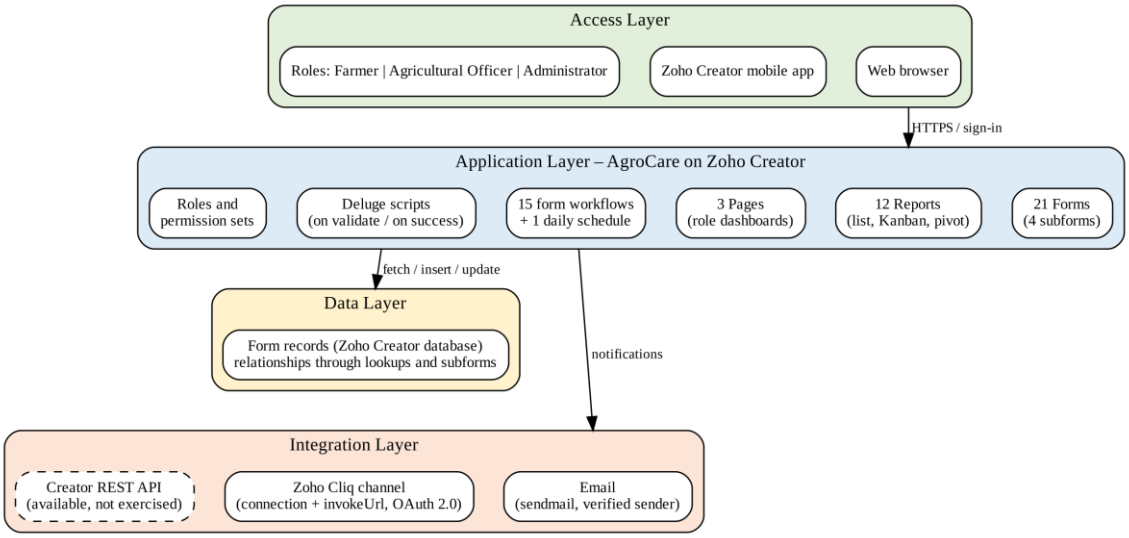


Figure 4.1: System Architecture of the Application

The architecture has four layers. The **access layer** consists of the web browser and the Zoho Creator mobile app used by the three roles. The **application layer** is the AgroCare app on Zoho Creator, made up of forms, reports, pages, Deluge scripts and workflows. The **data layer** holds the form records, with relationships implemented through lookups and subforms. The **integration layer** connects the application to Zoho Cliq through a Creator connection and the invokeUrl task, and to email through the sendmail task. A scheduled workflow runs every day to produce reminders, health scores and warnings.

4.2 Data Model

Figure 4.2 shows the entity–relationship diagram of the application, drawn from the forms and their lookup and subform links. In Zoho Creator, a one-to-many relationship is realised with a lookup field on the "many" side, and a parent–child relationship with a subform. Table 4.1 lists the main entities and relationships.

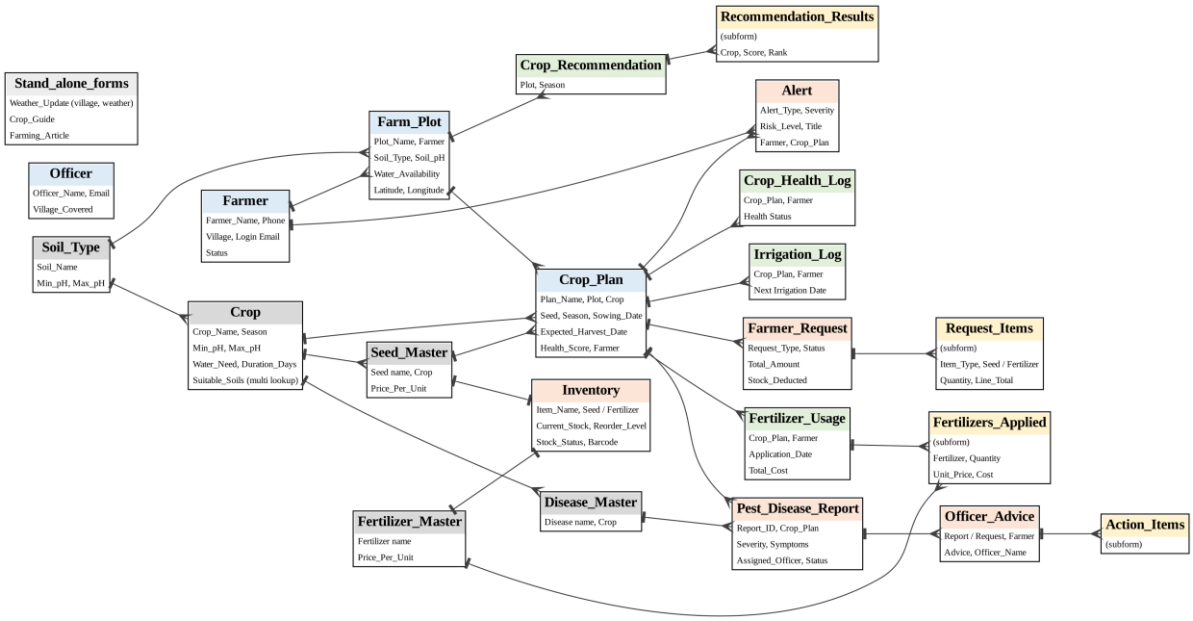


Figure 4.2: Entity–Relationship Diagram of the Application

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Soil_Type	Soil_Name, Min_pH, Max_pH	Farm_Plot, Crop	One-to-Many	Lookup
Crop	Crop_Name, Season, pH range, Water_Need, Duration_Days	Seed_Master, Disease_Master, Crop_Plan	One-to-Many	Lookup
Farmer	Farmer_Name, Phone, Village, Status	Farm_Plot	One-to-Many	Lookup in Farm_Plot
Farm_Plot	Plot_Name, Soil_pH, Water_Availability, Latitude, Longitude	Crop_Plan	One-to-Many	Lookup in Crop_Plan
Crop_Plan	Plan_Name, Season, Sowing_Date, Health_Score	Health, Irrigation, Fertilizer, Pest reports, Requests, Alerts	One-to-Many	Lookup
Fertilizer_Usage	Application_Date, Total_Cost	Fertilizers Applied	Parent–Child	Subform
Farmer_Request	Request_Type, Status, Total_Amount	Request Items	Parent–Child	Subform
Officer_Advice	Advice, Officer_Name	Action Items	Parent–Child	Subform
Crop_Recommendation	Plot, Season, Recommended_Crops	Recommendation Results	Parent–Child	Subform
Pest_Disease_Report	Severity, Symptoms, Status	Officer_Advice, Disease_Master	One-to-Many	Lookup
Inventory	Item_Name, Current_Stock, Reorder_Level	Seed_Master, Fertilizer_Master	One-to-One	Lookup

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Alert	Alert_Type, Severity, Risk_Level, Title	Farmer, Crop_Plan, Disease_Master	One-to-Many	Lookup

4.3 Form Design

The application has 21 forms. Table 4.2 groups them by module in the order in which they were built. Field specifications of two representative forms are given in Tables 4.3 and 4.4; the field types are named as they appear in the Zoho Creator form builder.

Table 4.2: Forms of the Application by Module

Module	Forms	Purpose
Master data	Soil_Type, Crop, Seed_Master, Fertilizer_Master, Disease_Master, Officer	Catalogue maintained by the administrator and used by lookups and scoring rules
Farmer side	Farmer, Farm_Plot, Crop_Plan, Crop_Recommendation	Farmer and plot registration, crop plans and ranked crop recommendations
Crop monitoring	Crop_Health_Log, Irrigation_Log, Fertilizer_Usage	Recording of crop activity, costs and inputs for the health score and reminders
Cases and warnings	Pest_Disease_Report, Officer_Advice, Alert, Weather_Update	Problem reporting, officer advice, alerts and weather-based advisories
Operations	Inventory, Farmer_Request	Seed and fertilizer stock, farmer requests and stock deduction
Knowledge	Crop_Guide, Farming_Article	Crop guides and farming articles published for farmers

Table 4.3: Field Specification – Farm_Plot Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Plot Name	Plot_Name	Single Line	Yes	—	Example: Murugan – North Field
Farmer	Farmer	Lookup (Farmer)	Yes	Dropdown	One farmer per plot
Soil Type	Soil_Type	Lookup (Soil_Type)	Yes	Dropdown	One soil type per plot
Plot Area (acres)	Plot_Area	Decimal	Yes	2 decimal places	
Soil pH	Soil_pH	Decimal	Yes	Must be 0–14	Used in crop scoring

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Water Availability	Water_Availability	Drop Down	Yes	Low / Medium / High	Matches crop Water_Need
Irrigation Source	Irrigation_Source	Drop Down	No	Borewell / Canal / Tank / Rainfed / Drip	
Latitude / Longitude	Latitude / Longitude	Decimal	Yes	6 decimal places	Location of the plot
Status	Status	Drop Down	Yes	Default: Active	Active / Inactive

Table 4.4: Field Specification – Pest_Disease_Report Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Report ID	Report_ID	Auto Number	—	System generated	
Crop Plan	Crop_Plan	Lookup (Crop_Plan)	Yes	Dropdown	Displays Plan Name
Farmer	Farmer	Lookup (Farmer)	No	Filled by workflow	
Report Date	Report_Date	Date	Yes	Default: today	
Suspected Disease	Suspected_Disease	Lookup (Disease_Master)	No	Dropdown	
Problem Category	Problem_Category	Drop Down	Yes	Fungal / Bacterial / Viral / Pest / Nutrient Deficiency / Not Sure	
Symptoms	Symptoms	Multi Line	Yes		
Severity	Severity	Drop Down	Yes	Low / Medium / High / Critical	
Photo	Photo	Image	No		
Assigned Officer	Assigned_Officer	Single Line	No	Filled by workflow	
Status	Status	Drop Down	Yes	Default: Submitted	Submitted / Under Review / Advisory Given / Monitoring / Resolved

Fields that are filled by workflows, such as Farmer and Assigned Officer above, and Plan Name, Unit Price, Cost, Total Cost and Health Score in other forms, are kept optional so that a record can be saved before the workflow runs.

4.4 Process and Workflow Design

Two processes carry most of the automation. Figure 4.3 shows pest and disease case handling: when a farmer submits a report, the Pest Report Setup workflow assigns an officer, sets the status to Under Review and notifies the officer by email and Cliq; when the officer saves advice, the Advice Follow Up workflow sets the status to Advisory Given and creates an alert for the farmer, after which the officer moves the case to Monitoring or Resolved.

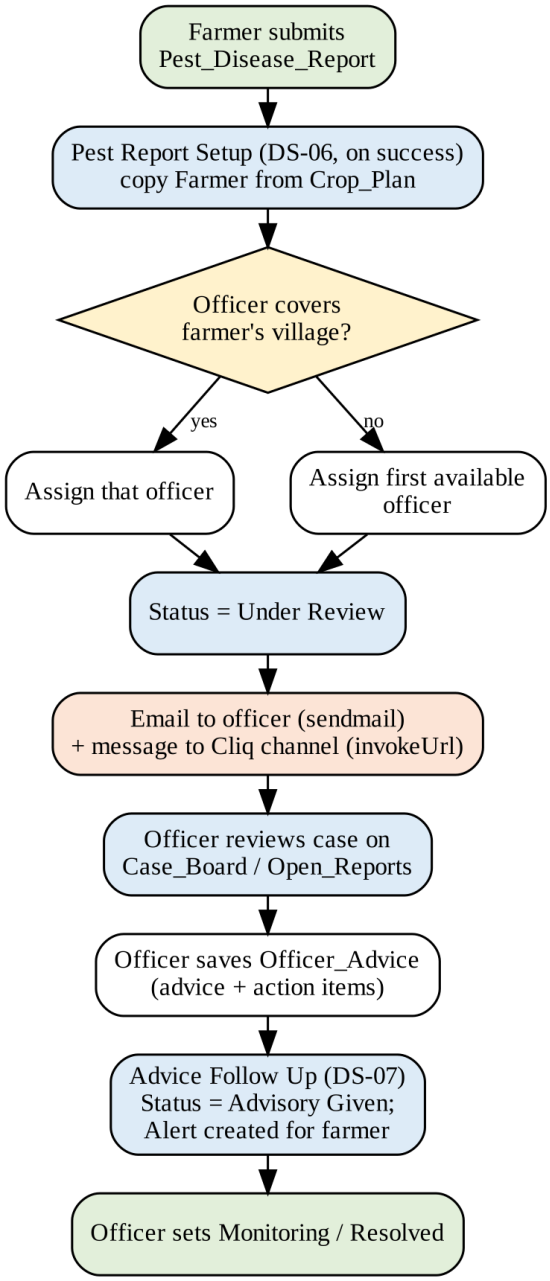


Figure 4.3: Workflow / Process Flow for Pest and Disease Case Handling

Figure 4.4 shows the request approval process. A request moves from Pending to Approved; on approval, the stock of every item is checked and either deducted or the

request is marked Out of Stock. The platform's Approvals and Blueprint features were not used; a Status field with a form workflow is simpler for this two-step flow.

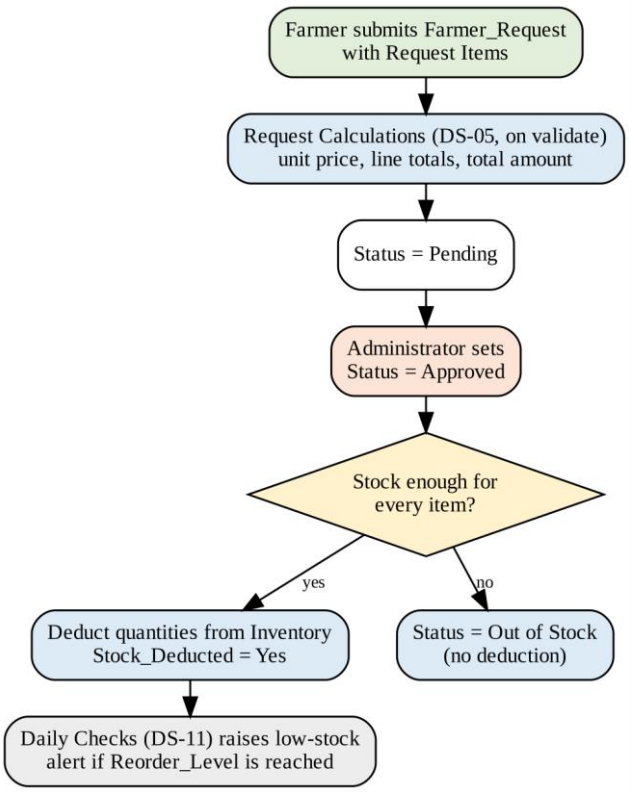


Figure 4.4: Workflow / Process Flow for Request Approval and Stock Deduction

4.5 User Interface and Navigation Design

The left menu is generated from the forms, reports and pages. Each role gets one landing dashboard: a green farmer dashboard showing active crops, health score, unread alerts and pending requests; a blue officer dashboard showing new, under-review, advisory-given and high-risk cases; and an administrator dashboard showing farmers, plots, active crops, open cases, pending requests and low-stock items, followed by embedded charts and lists. Figure 4.5 shows the navigation structure by role and Figure 4.6 the wireframe of the administrator dashboard.

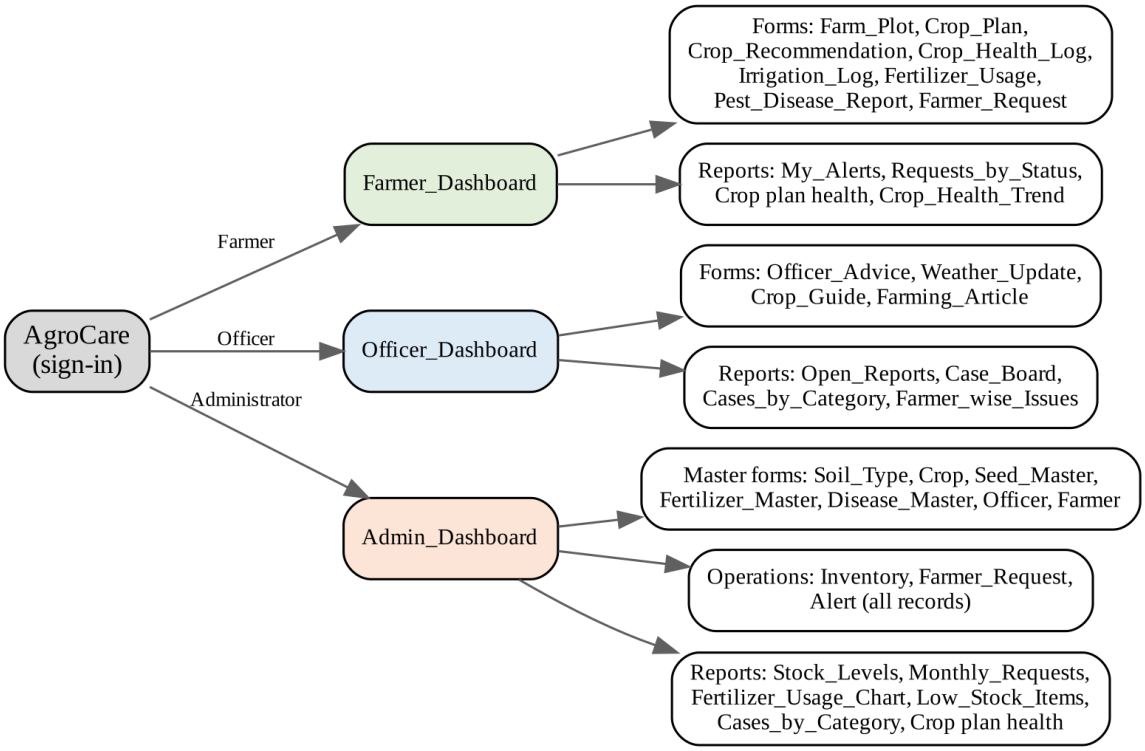


Figure 4.5: Navigation Structure of the Application by Role

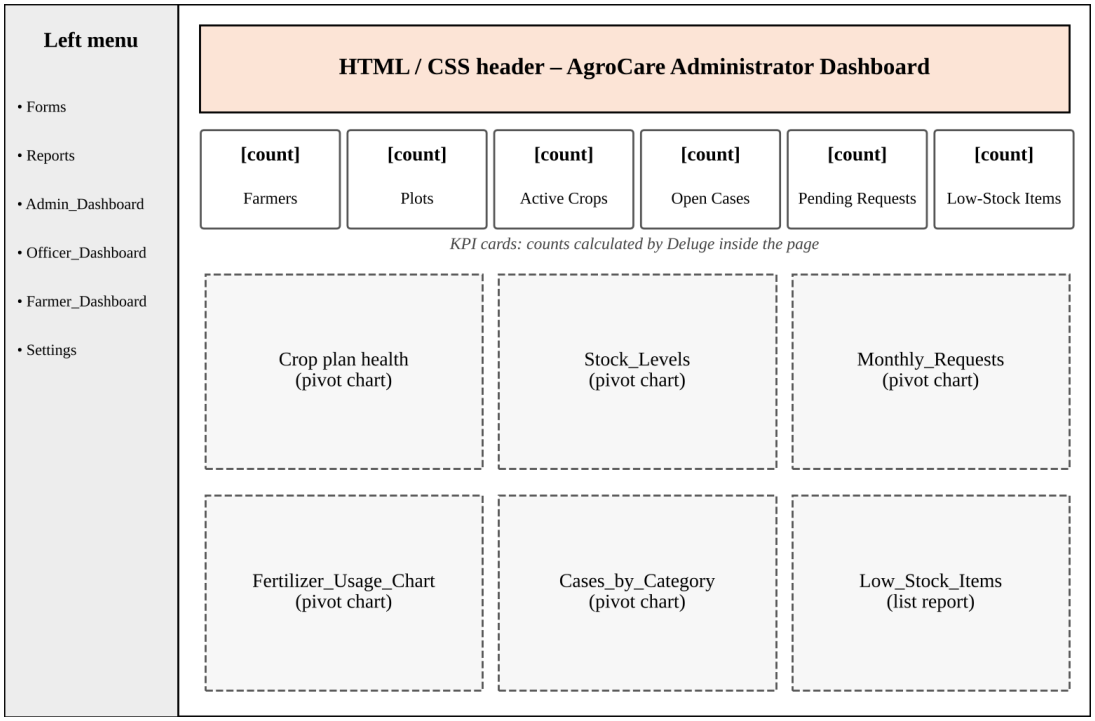


Figure 4.6: Wireframe of the Admin_Dashboard Page

4.6 Access Control Design

Table 4.5 defines the permission matrix that Section 5.8 implements (A = Add, E = Edit, V = View, D = Delete, — = No access).

Table 4.5: Role–Permission Matrix

Component	Administrator	Agricultural Officer	Farmer
Master forms (Soil, Crop, Seed, Fertilizer, Disease)	A E V D	V	V
Farmer, Officer, Weather_Update	A E V D	V (Weather: A V)	—
Farm_Plot, Crop_Plan	A E V D	V	A E V (own)
Crop_Health_Log, Irrigation_Log, Fertilizer_Usage	A E V D	V	A E V (own)
Pest_Disease_Report	A E V D	E V	A V (own)
Officer_Advice	A E V D	A E V	V (own)
Farmer_Request	A E V D	V	A V (own)
Inventory	A E V D	V	—
Alert	A E V D	A V	V (own)
Page: Admin_Dashboard	V	—	—
Page: Officer_Dashboard	V	V	—
Page: Farmer_Dashboard	V	—	V

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in Zoho Creator under the name AgroCare. The build followed the order: master forms, farmer-side forms, monitoring forms, operations forms, workflows, reports, dashboards, integrations and finally roles. The Localization setting was used to set the time zone and date format for India. Figure 5.1 shows the application home and its navigation sections.

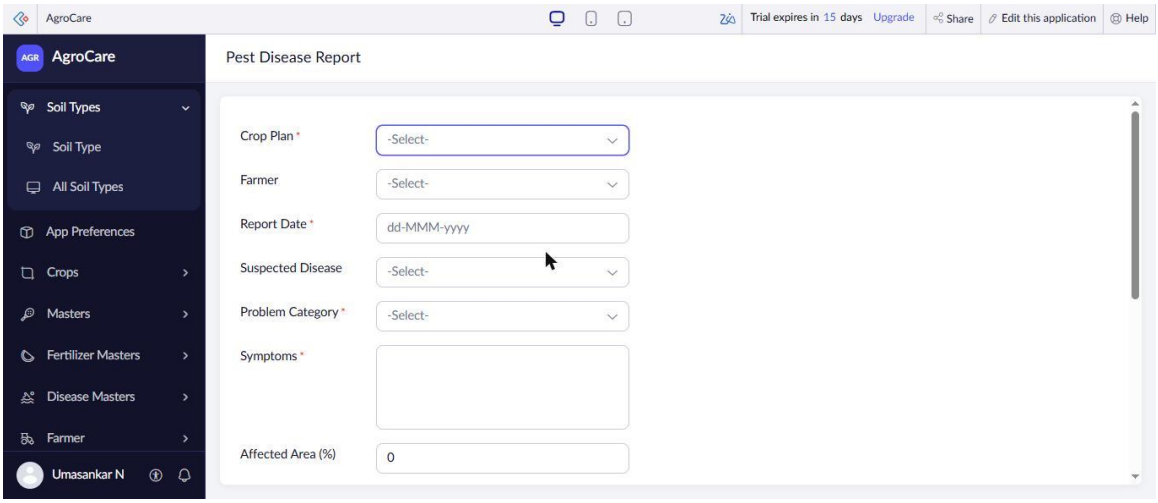


Figure 5.1: Application Left-Menu Navigation and the Pest Disease Report Form

5.2 Forms and Data Capture

The application has 21 forms using the field types Single Line, Multi Line, Number, Decimal, Currency, Date, Date-Time, Drop Down, Lookup, Subform, Email, Phone, Name, Address, Image, Auto Number and Barcode. Lookups connect forms, for example Crop_Plan to Farm_Plot, Crop and Seed_Master. A Lookup with multiple selection is used for suitable soils on the Crop form. Four subforms capture child records: Fertilizers Applied, Request Items, Action Items and Recommendation Results.

Mandatory fields, no-duplicate values (for example the farmer's phone number), default values (status fields) and the pH range check workflows provide validation. The Inventory form includes a Stock Status formula field and a Barcode field that generates a code for each item. Fields that are filled by workflows, such as Plan Name, Farmer in the logs, Unit Price, Cost, Total Cost and Health Score, are kept

optional so that a record can be saved before the workflow runs. Figures 5.2 and 5.3 show two of the forms in the Form Builder. Figure 5.3 shows the form in live mode.

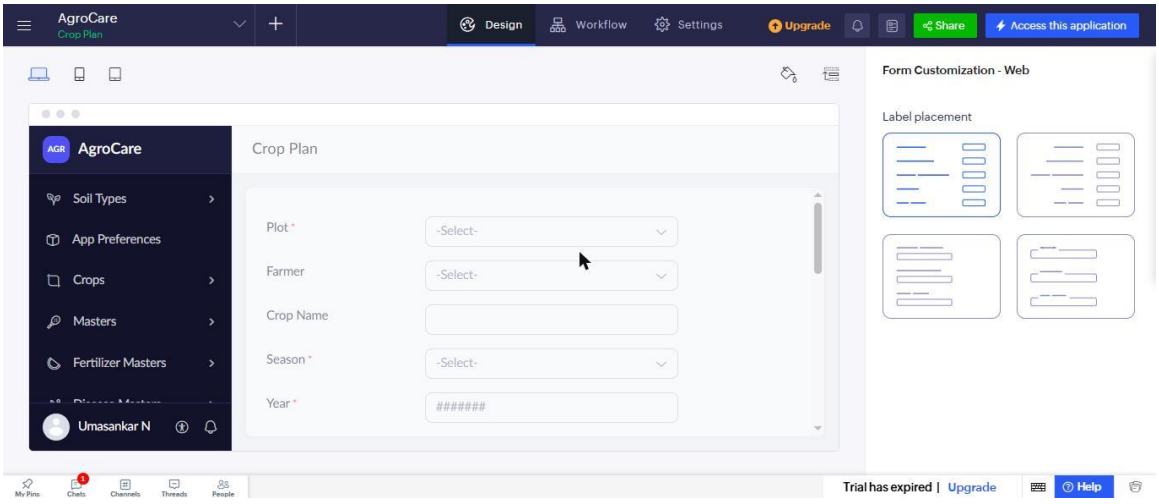


Figure 5.2: Crop_Plan Form in the Form Builder

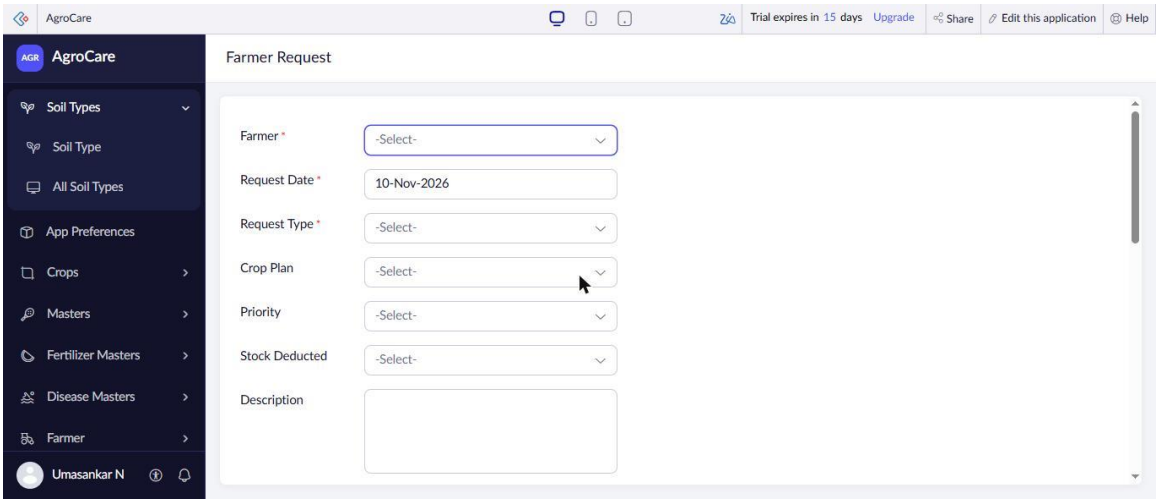


Figure 5.3: Farmer_Request Form in Live Mode

5.3 Reports

Twelve reports were built using the report types available in the account: list, Kanban, pivot chart and pivot table. Pivot charts group by text, dropdown and number fields, so where a lookup was not listed, a text field filled by a workflow was used instead. Table 5.1 lists the reports and Figure 5.4 shows the Case_Board Kanban report.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Roles
Crop plan health (Crop_wise_Area)	Pivot chart	Crop_Plan	Plan Name against Health Score	Farmer, Admin
Stock_Levels	Pivot chart	Inventory	Item Name against Current Stock	Admin

Report Name	Type	Source Form	Filter / Grouping / Sorting	Roles
Cases_by_Category	Pivot chart	Pest_Disease_Report	Problem Category, count	Officer, Admin
Monthly_Requests	Pivot chart	Farmer_Request	Request Date by month, count	Admin
Crop_Health_Trend	Pivot chart	Crop_Health_Log	Health Status, count	Farmer, Admin
Fertilizer_Usage_Chart	Pivot chart	Fertilizer_Usage	Crop Plan against Total Cost	Admin
Farmer_wise_Issues	Pivot table	Pest_Disease_Report	Rows: Farmer; Columns: Problem Category	Officer
Low_Stock_Items	List	Inventory	Stock Status = Low	Admin
My_Alerts	List	Alert	Sorted by Alert Date, newest first	Farmer
Open_Reports	List	Pest_Disease_Report	Status is not Resolved; sorted by Report Date descending	Officer
Requests_by_Status	Kanban	Farmer_Request	Lanes by Status	Admin, Farmer
Case_Board	Kanban	Pest_Disease_Report	Lanes by Status	Officer

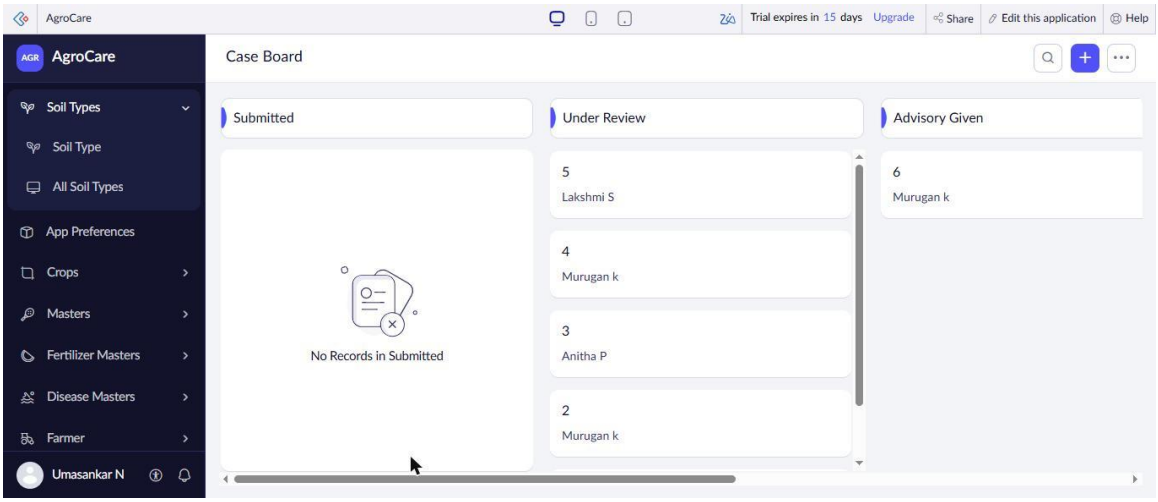


Figure 5.4: Case_Board (Kanban) Report with Case 6 in Advisory Given

5.4 Pages and Dashboards

Three pages were built. The Farmer_Dashboard embeds My_Alerts, Requests_by_Status, the crop plan health chart and the crop health trend. The Officer_Dashboard embeds Open_Reports, Case_Board, Cases_by_Category and Farmer_wise_Issues. The Admin_Dashboard embeds the crop plan health chart, Stock_Levels, Monthly_Requests, Fertilizer_Usage_Chart, Cases_by_Category and

Low_Stock_Items. Each page begins with an HTML and CSS header with KPI cards. Deluge inside the page counts records, for example total farmers, open cases and pending requests, and displays them as coloured cards. Figure 5.5 shows the Admin_Dashboard.

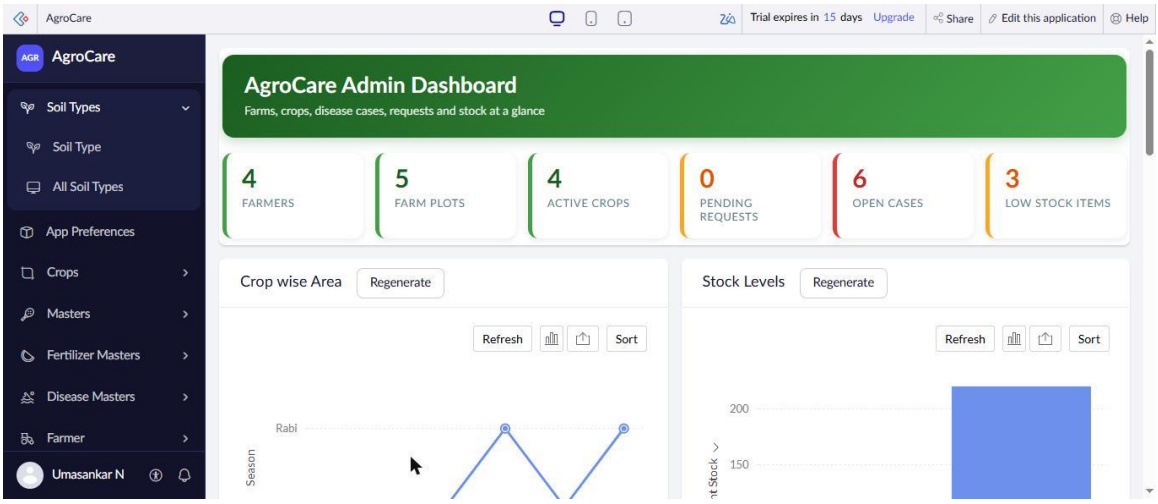


Figure 5.5: Admin_Dashboard Page with KPI Cards

5.5 Deluge Scripting

Deluge scripts [6] run on form events (on validate and on success), in schedules and in page code. They use fetch statements with criteria, insert into form, for each loops, collections (List and Map), conditional logic, date functions and the sendmail and invokeUrl tasks. Table 5.2 lists the scripts; the complete code is given in Appendix C.

Table 5.2: Deluge Scripts Implemented

Script ID	Name	Runs On	Purpose
DS-01	Fill Plan Name	Crop_Plan, create/edit	Builds Plan Name as Plot - Crop - Season and copies the Farmer from the plot
DS-02	Calculate Harvest Date	Crop_Plan, on validate	Expected harvest = sowing date + crop duration
DS-03	Fill Farmer (logs)	Crop_Health_Log, Irrigation_Log, on validate	Copies Farmer from the crop plan
DS-04	Fertilizer Usage Calculations	Fertilizer_Usage, on validate	Copies price, calculates row cost and total cost
DS-05	Request Calculations	Farmer_Request, on validate	Copies price, calculates line totals, total amount and suggested inputs
DS-06	Pest Report Setup	Pest_Disease_Report, on success	Assigns officer, sets status, sends email and Cliq message
DS-07	Advice Follow Up	Officer_Advice, on success	Sets status to Advisory Given, creates alert and emails the farmer

Script ID	Name	Runs On	Purpose
DS-08	Recommend Crops	Crop_Recommendation, on validate	Scores crops out of 100 and ranks them
DS-09	Approve Request and Deduct Stock	Farmer_Request, on edit	Checks stock, deducts it, or marks Out of Stock
DS-10	Weather Advisory	Weather_Update, on success	Creates rain, heat and disease-risk alerts for farmers in the village
DS-11	Daily Checks	Schedule, daily	Low stock, health score, reminders, community early warning
DS-12	pH range checks	Soil_Type, Crop, on validate	Rejects invalid or reversed pH values
DS-13	Dashboard KPI cards	Pages	Counts records and shows KPI cards

The crop suitability score (DS-08) is the main decision-support rule. The snippet below shows how the four criteria add up to 100.

```
// DS-08 Recommend Crops (extract) – Soil 35 + Season 30 + Water 20 +
pH 15
score = 0;
if(c.Suitable_Soils.contains(plot.Soil_Type)) { score = score + 35; }
if(c.Season == input.Season) { score = score + 30; }
if(waterDiff == 0) { score = score + 20; }
else if(waterDiff == 1) { score = score + 10; }
if(plot.Soil_pH >= c.Min_pH && plot.Soil_pH <= c.Max_pH) { score =
score + 15; }
```

5.6 Workflows and Automation

Fifteen form workflows and one scheduled workflow were created (Table 5.3). Form workflows keep data consistent and trigger notifications. The schedule runs daily and performs jobs that depend on dates and on records in several forms. The platform's Approvals and Blueprint features were not used; request approval is handled by a Status field with a form workflow, which is simpler for a two-step flow. Figure 5.6 shows the configuration of the Pest Report Setup workflow.

Table 5.3: Workflows Implemented

No.	Workflow	Type	Form	Trigger	Action(s)	Script Ref.
1	Fill Plan Name	Form	Crop_Plan	Create/Edit	Plan Name, Crop Name, Farmer	DS-01
2	Calculate Harvest Date	Form	Crop_Plan	On validate	Expected harvest date	DS-02
3	Fill Farmer in Health Log	Form	Crop_Health_Log	On validate	Copy farmer	DS-03

No.	Workflow	Type	Form	Trigger	Action(s)	Script Ref.
4	Fill Farmer in Irrigation Log	Form	Irrigation_Log	On validate	Copy farmer	DS-03
5	Fertilizer Usage Calculations	Form	Fertilizer_Usage	On validate	Prices and totals	DS-04
6	Request Calculations	Form	Farmer_Request	On validate	Prices, totals, suggestions	DS-05
7	Pest Report Setup	Form	Pest_Disease_Report	Create, on success	Assign officer, email, Cliq	DS-06
8	Advice Follow Up	Form	Officer_Advice	Create, on success	Status, alert, email	DS-07
9	Fill Inventory Item Name	Form	Inventory	On validate	Item name	—
10	Soil pH Check	Form	Soil_Type	On validate	Reject invalid pH	DS-12
11	Crop pH Check	Form	Crop	On validate	Reject invalid pH	DS-12
12	Recommend Crops	Form	Crop_Recommendation	On validate	Score and rank crops	DS-08
13	Approve Request and Deduct Stock	Form	Farmer_Request	Edit, on success	Stock deduction	DS-09
14	Low Stock Alert	Form	Inventory	Create/Edit, on success	Alert and email	—
15	Weather Advisory	Form	Weather_Update	Create, on success	Weather and disease-risk alerts	DS-10
16	Daily Checks	Schedule	—	Daily 6:00 AM	Low stock, health score, reminders, early warning	DS-11

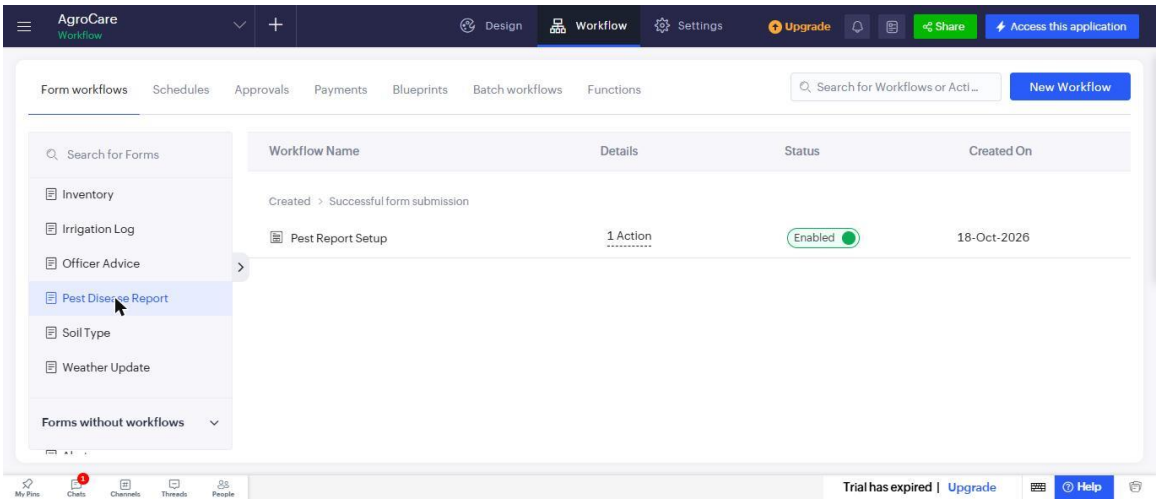


Figure 5.6: Pest Report Setup Workflow on Pest_Disease_Report (enabled)

5.7 Integrations

Email notifications use the sendmail task from a verified sender address. The Zoho Cliq integration uses a connection created under Developer tools with the Cliq webhook scope [7]. The Pest Report Setup workflow posts a message to a Cliq

channel through invokeUrl, referring to the connection by its link name, so no token or password is written in the script. A live weather API call through invokeUrl was designed but replaced by manual weather entry in this version. Every form is also reachable through the Creator REST API provided by the platform; this was not exercised. Table 5.4 summarises the integrations and Figure 5.7 shows the workflow area where the form workflows, schedule and functions are managed.

Table 5.4: Integrations Implemented

Service	Method	Authentication	Data Exchanged	Used In
Zoho Cliq	Connection + invokeUrl POST	OAuth 2.0 (Creator connection)	New case message to the officers' channel	Pest Report Setup
Email	sendmail task	Verified sender address	Case assignment, advice, reminders, early warning	Workflows 7, 8, 14 and the schedule
Creator REST API	Built-in	OAuth 2.0	Records of forms (available, not exercised)	Optional

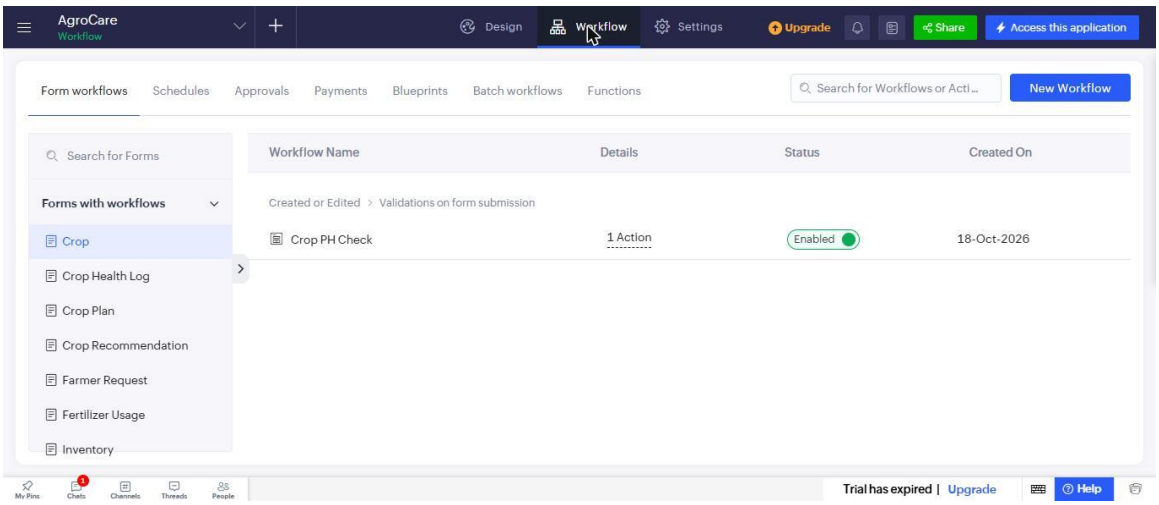


Figure 5.7: Workflow Area: Form Workflows, Schedules and Functions (Crop PH Check enabled)

5.8 User Management and Role-Based Access

The role hierarchy in the application has the top role CEO (held by the application owner, who acts as Administrator) with two child roles, Agricultural Officer and Farmer (Figure 5.8). Two permission profiles exist in the application, Read and Write, which grant read or write permission on all components, and users are assigned a role and a profile. Table 4.5 shows the intended access for each role. Farmers are linked to their records through a Login Email field on the Farmer form, which is used in the dashboard script.

Limitation: custom component-level permission sets and record-level criteria per role were not configured in this version; the Read and Write profiles apply to all components. Restricting each role to its own records is therefore listed as future work.

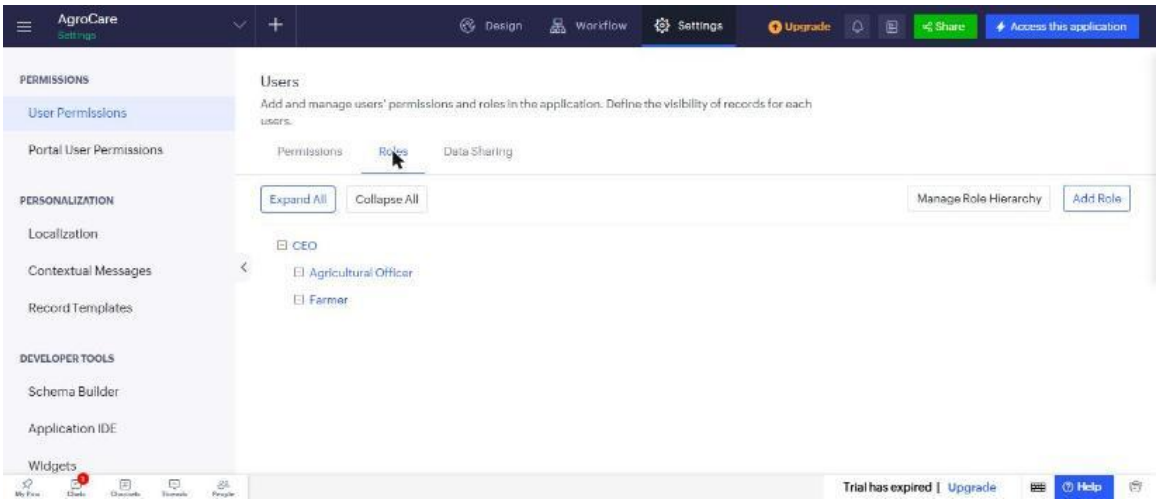


Figure 5.8: Roles Hierarchy: CEO, Agricultural Officer and Farmer

5.9 AI and Analytics Features

No Zia or machine-learning feature was used. Decision support is rule-based and implemented in Deluge:

- **Crop suitability score** – soil 35, season 30, water 20 and pH 15, with partial credit for adjacent water levels and for pH within 0.5 of the range.
- **Crop health score** – irrigation in the last 7 days 25, fertilizer in the last 30 days 20, no open pest report 25, no disease-risk alert in 7 days 15, and a good health log in 14 days 15.
- **Disease-risk level** – Low, Medium or High from open reports.
- **Community early warning** – a high-risk alert when three or more farmers in a village report the same disease.

These rules were written by the student and verified against hand-calculated examples; for instance, Rice on the plot Murugan – North Field (pH 6.8, high water, Kharif season) scores 65 out of 100 in a test run on 10 November 2026. The Soil Type column of the recommendation record stays blank, so the 35 soil points are not awarded; this is logged as defect D-13, and with the soil match working the same rule would give 100. Analytics are provided by pivot charts, Kanban boards and KPI dashboards. Figure 5.9 shows the ranked crop recommendation output.

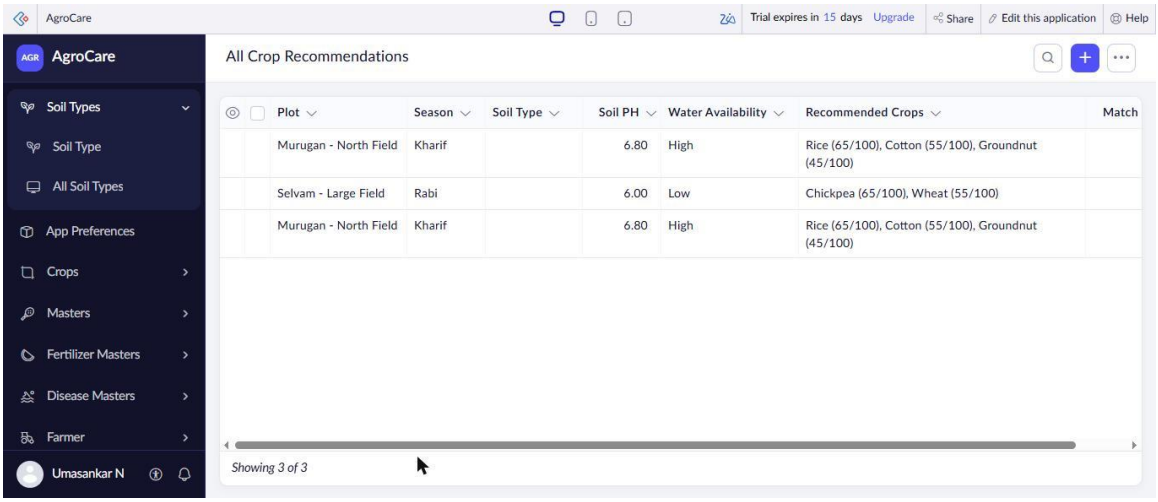


Figure 5.9: Ranked Crop Recommendation Output (All Crop Recommendations report)

5.10 Pro-Code Extension

No pro-code extension such as Zoho Catalyst or JavaScript widgets was used, because the requirements were met with Deluge and platform features. In future, a widget could show weather charts, Catalyst could host an image-based disease detection model, and the Creator REST API could connect a farmer mobile app or an SMS gateway.

5.11 Mobile and Portal Access

The application can be used through the Zoho Creator mobile app, where forms, reports and dashboards are available, and the Barcode field can generate item codes. A customer portal was not used; farmers are added as application users with the Farmer role. In a production deployment a portal would be the better choice for external farmers.

Mobile access was not tested in this version.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was planned at four levels. **Unit testing** checked each form (mandatory fields, lookups, defaults) and each script on its own with sample data. **Integration testing** followed a record through several forms and workflows, for example a pest report that triggers officer assignment, email and a Cliq message, and later an advice record that updates the report status. **Role-based testing** was done by logging in as each role. **User acceptance testing** was done with two or more end users. Scheduled jobs were tested with the schedule's Execute Now option.

6.2 Test Cases

Table 6.1 lists the test cases. The expected values come from the sample data used in the application.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add farmer with duplicate phone	Phone 9876543210 twice	Second record rejected	Second farmer with the same phone rejected: "This value already exists. Enter a unique value."	Pass
TC-02	FR-02	Save soil with Min pH 8 and Max pH 6	Min 8, Max 6	Alert shown, record not saved	Alert "Min pH cannot be greater than Max pH" shown; record not saved	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-03	FR-04	Save a crop plan for Murugan – North Field, Rice, Kharif	Sowing 15-Aug-2026	Plan Name = Murugan - North Field - Rice - Kharif; harvest 13-Dec-2026	Existing plan Murugan - North Field - Rice - Kharif (sowing 15-Aug-2026) has the expected plan name	Pass
TC-04	FR-03	Recommend crops for Murugan – North Field, Kharif	Plot, Season	Rice ranked first (live data: Rice 65/100, Cotton 55/100, Groundnut 45/100)	Rice ranked first with 65/100, then Cotton 55 and Groundnut 45; Soil Type stayed blank (defect D-13)	Pass (score 65, not 100)
TC-05	FR-05	Save fertilizer usage with Urea 50 and DAP 40	Two subform rows	Costs 13,500 and 54,000; total 67,500	Urea 50 saved; Total Cost 13500.0000 (Usage record 3)	Pass
TC-06	FR-07	Submit a pest report for a Thovalai farmer	Severity Medium	Farmer filled, officer for Thovalai assigned, status Under Review, email sent	New report 6 saved: Farmer filled (Murugan k), Officer Ravi assigned, status Under Review	Pass
TC-07	FR-16	Submit a pest report	Any	Message appears in Cliq channel	Cliq channel is outside this session; the workflow saved the report without error	Not verified

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-08	FR-08	Save officer advice linked to a report	Advice text	Report status Advisory Given; alert created for farmer	Advice saved for report 6; Case_Board moved case 6 to Advisory Given; Alert 5 "Officer advice received" created for Murugan k	Pass
TC-09	FR-12	Save request with Cotton Seed Bt ×2 and DAP ×3	Subform rows	Line totals 1,700 and 4,050; total 5,750	Cotton Seed Bt x 2 saved; Total Amount 1700.00 (DAP line not tested); Status stayed blank (defect D-15)	Pass (partly)
TC-10	FR-13	Approve a request with enough stock	Status Approved	Stock reduced by the requested quantity	Request 4 set to Approved: Approved By and date filled, Stock Deducted = Yes; both Cotton Seed Bt inventory rows fell from 40 to 38 (defect D-16)	Pass (stock deducted twice)
TC-11	FR-13	Approve a request exceeding stock	Quantity above stock	Status Out of Stock; stock unchanged	Not run: it would need stock below the requested quantity and would change the demo inventory	Not run

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-12	FR-14	Run daily schedule with Cotton Seed Bt below reorder level	Execute Now	Low-stock alert created	Admin_Dashboard shows 3 low-stock items; schedule is disabled and no low-stock alert exists in All_Alerts	Not run (schedule disabled)
TC-13	FR-11	Run schedule with overdue Next Irrigation Date	Execute Now	Irrigation due alert and email	Daily schedule is disabled in the account; enabling it would send real emails	Not run (schedule disabled)
TC-14	FR-06	Run schedule for active crop plans	Execute Now	Health Score and Rating filled	Daily schedule is disabled in the account	Not run (schedule disabled)
TC-15	FR-09	Add three Rice Blast reports from Thovalai within 7 days and run schedule	Execute Now	High-risk community alert for Thovalai farmers	Daily schedule is disabled in the account	Not run (schedule disabled)
TC-16	FR-10	Add weather for Thovalai: 27 °C, 88 % humidity	Weather_Update record	Rice Blast risk alert for farmers with active Rice plans	Weather record Thovalai 27 °C, 88 % saved but Alerts Created = 0 and no new alert (defect D-14)	Fail
TC-17	FR-17	Log in as Farmer and open Inventory	—	Access denied or form not visible	Custom role permissions are not configured (Read and Write profiles only)	Not applicable
TC-18	FR-17	Log in as Farmer and view alerts	—	Only own alerts visible	Record-level criteria are not configured	Not applicable

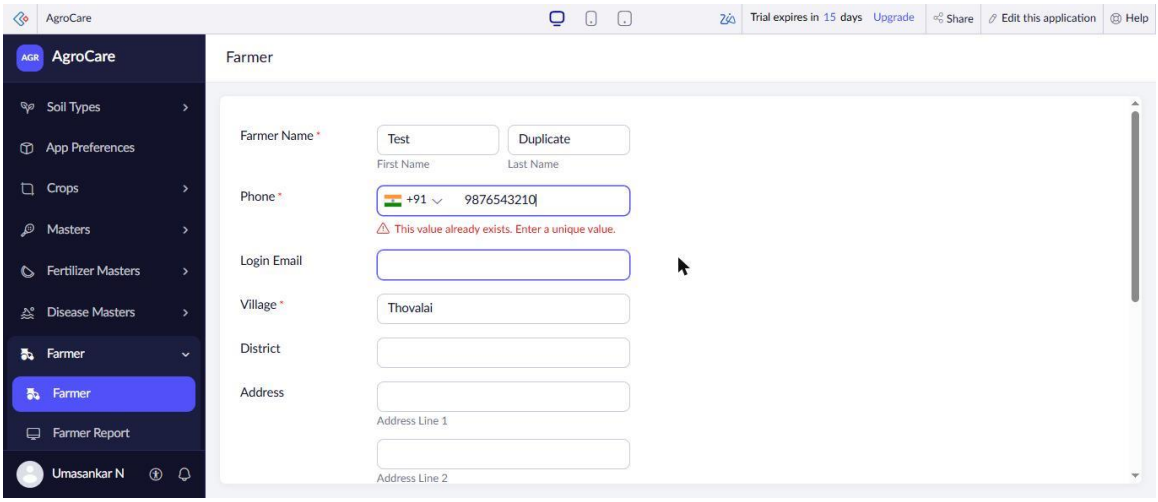


Figure 6.1: TC-01: Duplicate Phone Number Rejected on the Farmer Form

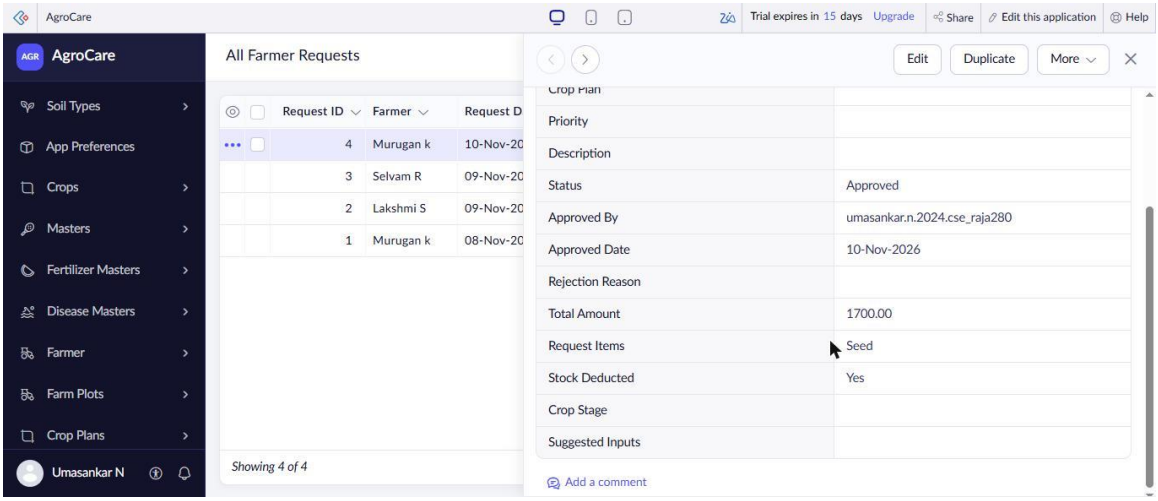


Figure 6.2: TC-10: Request 4 Approved, Stock Deducted = Yes

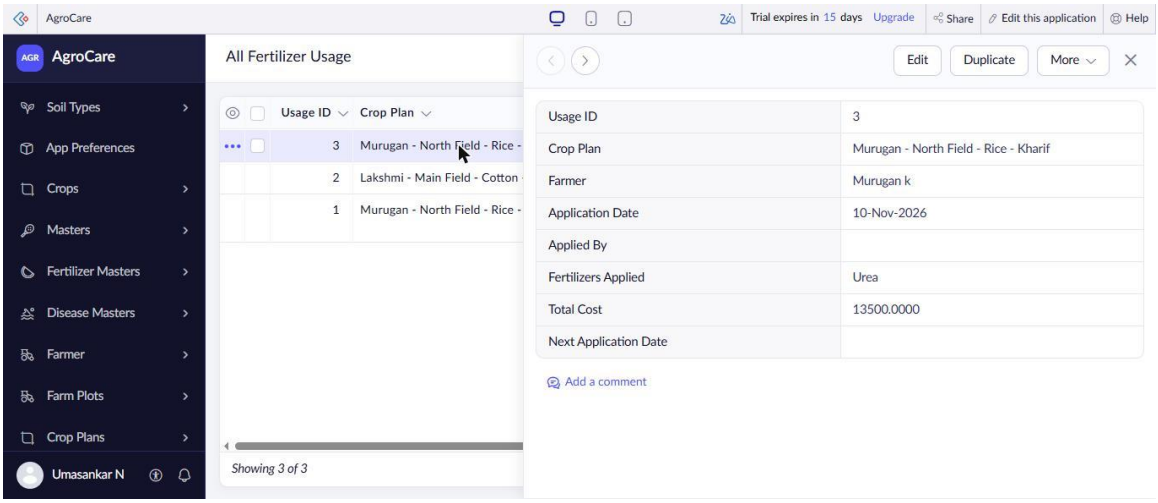


Figure 6.3: TC-05: Fertilizer Usage Record with Total Cost 13500

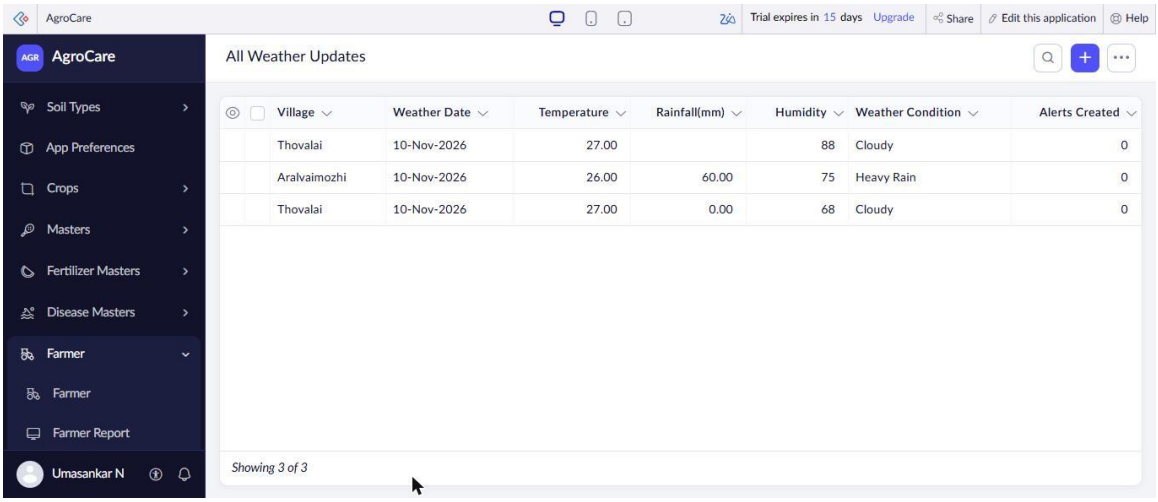


Figure 6.4: TC-16: Weather Updates Report (Alerts Created = 0)

6.3 Debugging Deluge Scripts

Errors were found through the Deluge editor's messages when saving a script, through errors shown on form submission, and through info statements and execution logs. The commonest causes were link names that did not match the form (for example a field named differently from the script), fields of the wrong type (a formula or text field written with a number), and functions not available in the version used.

Two real cases illustrate the approach. The Fertilizer Usage workflow showed "Total cost cannot be updated" because Total Cost had been created as a Formula field; changing it to a Decimal field fixed it. The Cliq message did not arrive because the script used a channel name that differed from the channel's unique name; correcting the name and capturing the response with an info statement resolved it. Figure 6.1 shows an error and the execution log.

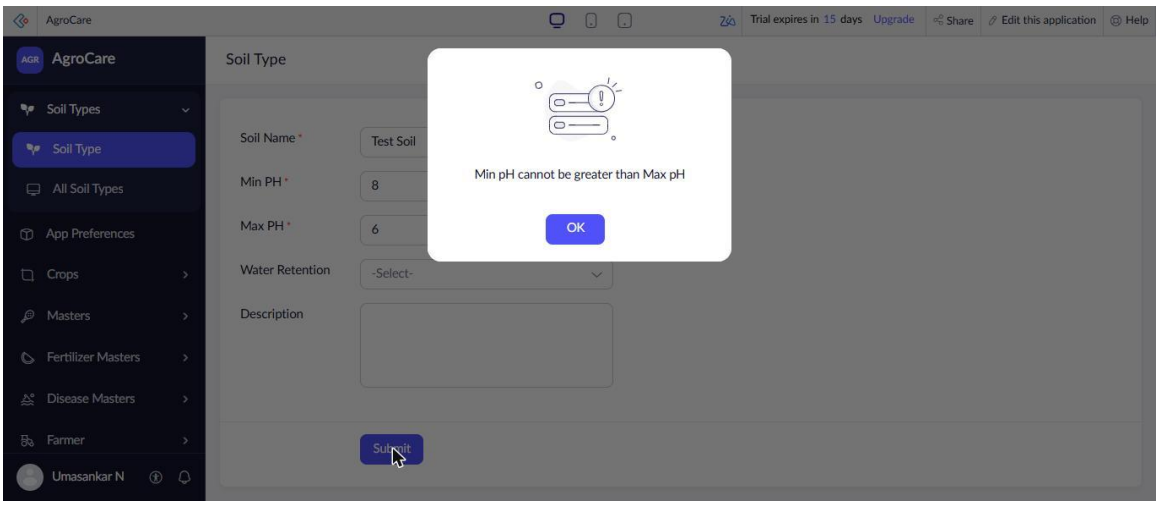


Figure 6.5: Validation Message from the pH Range Check Script (Min pH cannot be greater than Max pH)

6.4 Defect Log

Table 6.2: Defect Log

ID	Defect	Cause	Fix	Status
D-01	Multi Select field had no lookup option	Plain Multi Select takes typed choices only	Used a Lookup field with multiple selection	Closed
D-02	Syntax error in Fill Plan Name script (missing ;)	Direct assignment to a fetched record expression	Fetched the record into a variable, then updated its fields	Closed
D-03	Formula field could not be added inside a subform	Platform limitation	Used a Decimal field and filled it with a workflow	Closed
D-04	"Fertilizers_Applied is not defined"	Subform link name differed from the script	Corrected the link name	Closed
D-05	"Expecting string but found decimal"	Target field was a text field	Changed Cost and Unit Price to Decimal	Closed
D-06	"Total cost cannot be updated"	Total Cost was a Formula field	Changed to Decimal	Closed
D-07	"Officer does not exist"	Officer form not yet created	Created the Officer form	Closed
D-08	"Invalid collection object" and "first function not found"	Functions unavailable for fetched collections	Used a for each loop to take the first record	Closed
D-09	while loop rejected in the crop score script	Deluge has no while loop	Replaced with for each and a used-list	Closed
D-10	"Admin email in from address not specified"	zoho.adminuserid did not resolve	Used a verified sender address	Closed
D-11	Pivot chart did not list lookup or some dropdown fields	Report type limitation	Used workflow-filled text fields or other available fields	Closed
D-12	Cliq message not delivered	Wrong channel unique name	Used the channel's unique name	Closed
D-13	Soil Type blank in recommendation record; soil points not awarded (Rice 65, not 100)	Soil_Type not copied from the plot in the Recommend Crops script	Copy the plot's soil type into the record before scoring	Open
D-14	Weather Advisory created 0 alerts for Thovalai 27 °C, 88 % humidity	Disease_Master temperature and humidity ranges did not match, or the rainfall field was blank	Check Disease_Master ranges and make Rainfall default to 0	Open
D-15	New Farmer_Request saved with blank Status and Priority	No default value on the Status field	Set the default Status to Pending	Open
D-16	Approval deducted 4 packets for a request of 2 (both duplicate Cotton Seed Bt rows reduced)	Two Inventory rows exist for the same seed	Remove the duplicate row and add a unique check on Seed	Open

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Played	Tasks Performed	Feedback	Rating (1–5)
Umasankar N (developer)	Administrator (owner)	Opened reports, Case_Board, Admin_Dashboard and recommendations	Counts matched the sample records; one chart showed No Data Available	Self-review, not rated
External users	Farmer, Agricultural Officer	Not conducted	No external users took part in this version; listed as future work	Not rated

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed in the development environment and tested with sample data. The trial account provides a single environment, so the Stage and Production environments were not used and the application was published for access by its URL only within the trial. The application was later imported into a second trial account, which is the version described and tested in this report. Before publishing, all workflow scripts were saved without errors, sample data was checked, and roles were tested. A rollback is possible by restoring the previous version or by exporting the application as a backup file before changes.

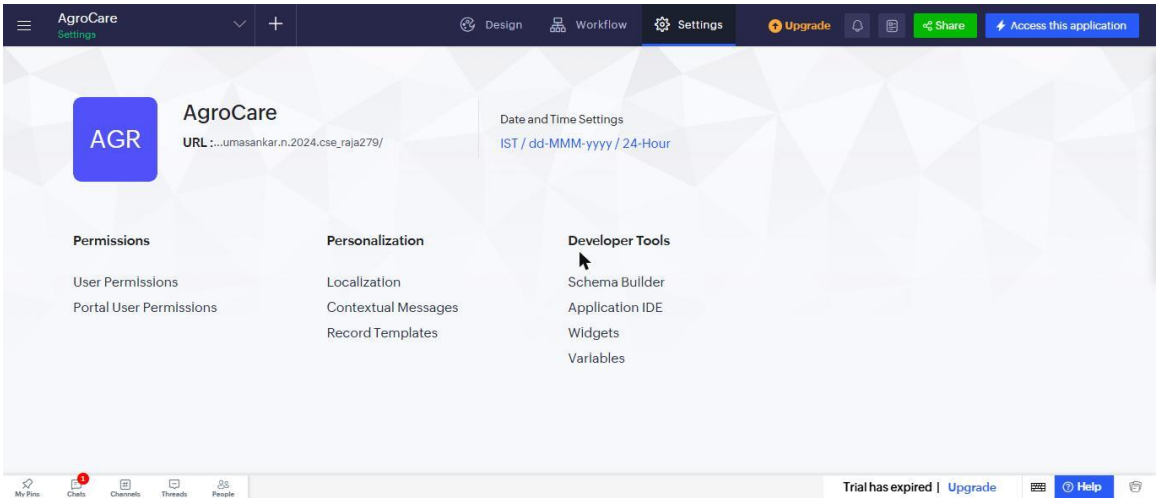


Figure 7.1: Application Settings: URL, Permissions and Developer Tools

Table 7.1: Deployment Checklist

S.No.	Item	Status
1	All forms saved and link names verified	Done
2	All workflows saved without errors	Done (workflows enabled)
3	Sender address verified	Not verified
4	Sample data loaded and tested	Done (5 plots, 4 farmers, 6 crop plans)
5	Roles and permissions tested for each role	Not done
6	Cliq connection authorised	Used by Pest Report Setup; not re-tested
7	Application exported as backup	Not done

S.No.	Item	Status
8	Published and URL shared	URL available (trial edition)

7.2 Performance Optimisation

Fetch statements use criteria so that only the needed records are read, for example active plans only. Duplicate-alert checks prevent repeated records. Workflows that need data from several forms are placed in one scheduled job that runs once a day instead of on every save. Reports show only the necessary columns and are filtered; for example Open_Reports excludes resolved cases. Fetches inside loops exist in some scripts (for example the early warning); with the current data volume this is acceptable, and these can be restructured with collections for larger data.

Table 7.2: Performance Optimisation Results

Technique	Where Applied	Observed Effect
Criteria in fetch statements	All workflows	Not timed; reports opened within the 8 s wait used in testing
Daily schedule instead of per-save logic	Health score, reminders, early warning	Not measured; schedule is disabled
Duplicate-alert checks	Schedule, low stock	Not timed; no duplicate alerts seen in All_Alerts
Filtered reports	Open_Reports, Low_Stock_Items	Each report opened within the 8 s wait used in testing; no timing tool was used

7.3 Security Features

Access is controlled by roles and permission sets. The Cliq connection uses OAuth 2.0 through a Creator connection, so no token is stored in a script. Email is sent only from a verified sender. Farmers are limited to their own records through the Login Email field. Sensitive items such as Inventory and Officer details are hidden from farmers. Two-factor authentication and IP restrictions were not configured in this version.

Table 7.3: Security Controls Implemented

Control	Implementation	Status
Role-based permissions	Administrator, Agricultural Officer, Farmer	Partly (roles defined; Read/Write profiles)

Control	Implementation	Status
Record-level access	Farmer's Login Email field available; criteria not configured	Not done
OAuth connection	Cliq connection, no hard-coded token	Done
Input validation	Mandatory fields, duplicate check, pH range check	Done
Page and report visibility	Dashboards and reports available in the left menu to permitted users	Partly

7.4 Governance and Administration

The audit trail records changes to data, and the platform keeps application and user management in one place. Names of forms, workflows and fields follow a consistent convention: forms in Capitalised_Underscore, and workflows named by purpose such as Fill Plan Name and Approve Request and Deduct Stock. The application can be exported as a backup, and user licences are managed by the administrator. Maintenance responsibilities are divided so that the administrator maintains master data and users, and the application owner maintains scripts. Figure 7.2 shows the audit trail and administration settings.

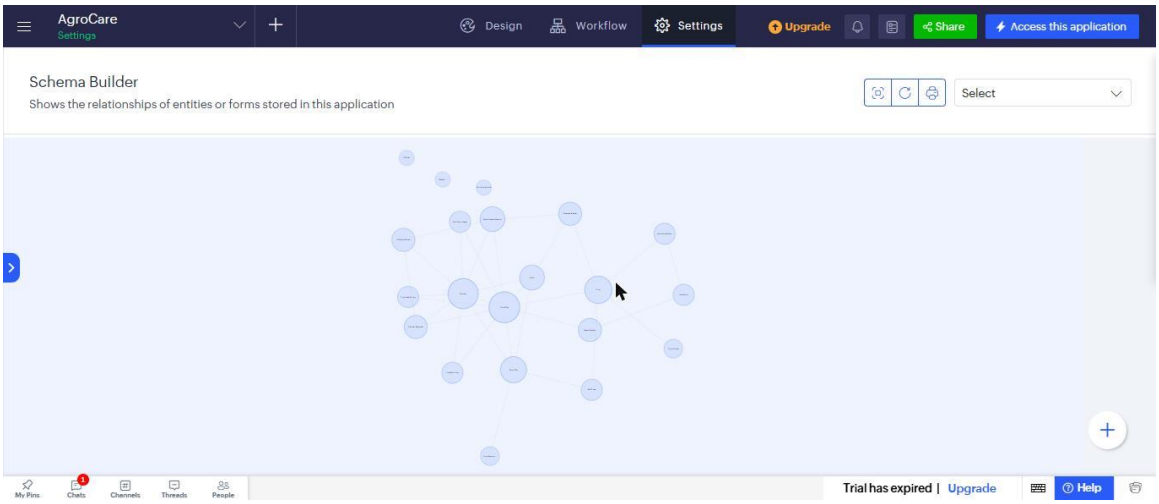


Figure 7.2: Schema Builder: Relationships Between the Application Forms

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below follows one case through the three roles.

- **Step 1** – Farmer selects the plot Murugan – North Field and the Kharif season, and the system ranks Rice first (65/100 in the live record, followed by Cotton 55 and Groundnut 45) (Figure 8.1).
- **Step 2** – Farmer records a crop plan, logs irrigation and fertilizer use, and reports leaf spots on the rice crop. The system assigns the Thovalai officer, sets the status to Under Review, sends an email and posts to Cliq (Figure 8.2).
- **Step 3** – Officer opens the case from the case board and adds advice with action items; the status becomes Advisory Given and the farmer gets an alert (Figure 8.3).
- **Step 4** – Farmer requests Urea; the administrator approves it and the stock reduces; the dashboard counts update (Figure 8.4).

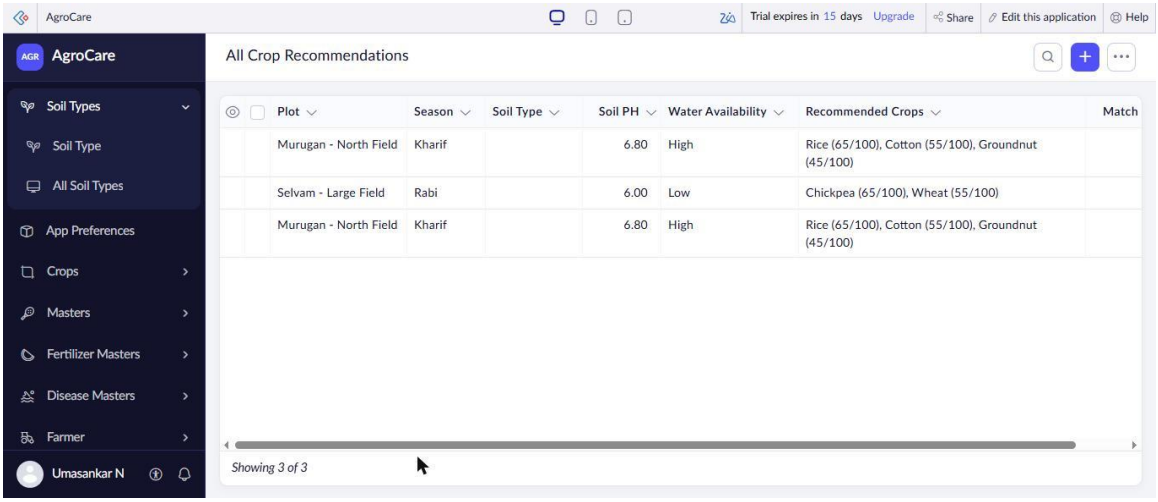


Figure 8.1: Scenario Step 1 – Crop Recommendation with Suitability Score

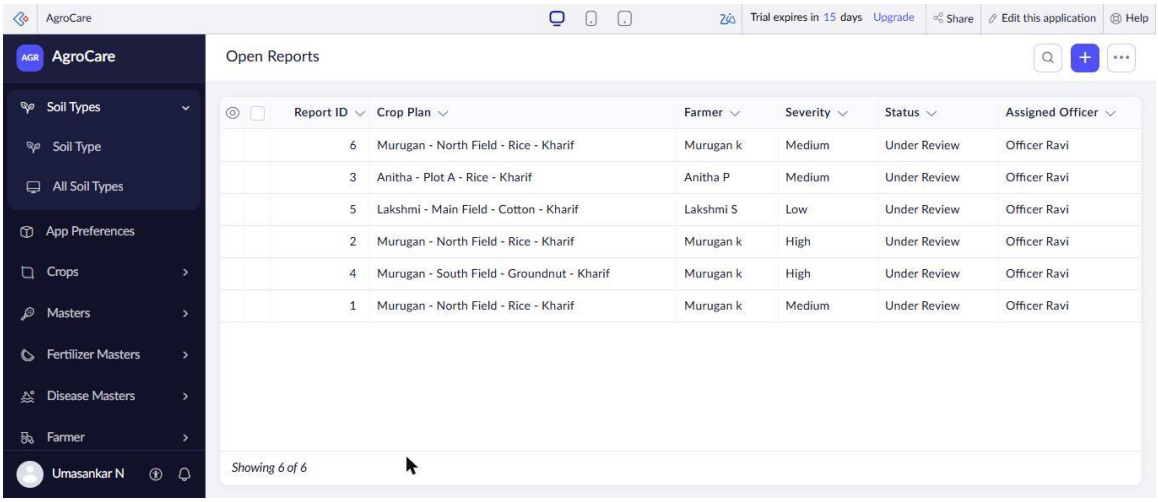


Figure 8.2: Scenario Step 2 – Pest Report 6 Submitted: Farmer Filled, Officer Ravi Assigned, Under Review

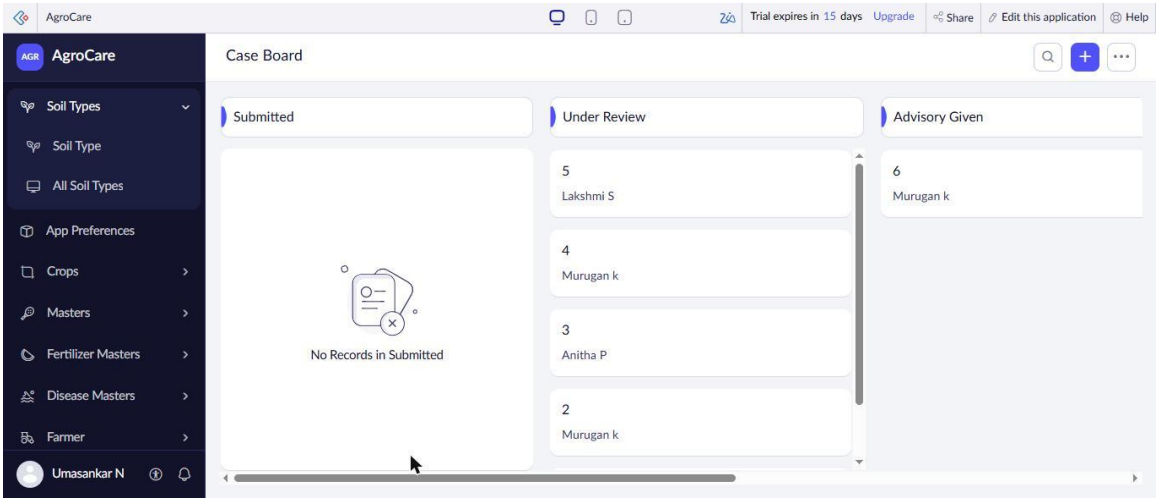


Figure 8.3: Scenario Step 3 – Officer Advice Saved; Case 6 Moves to Advisory Given

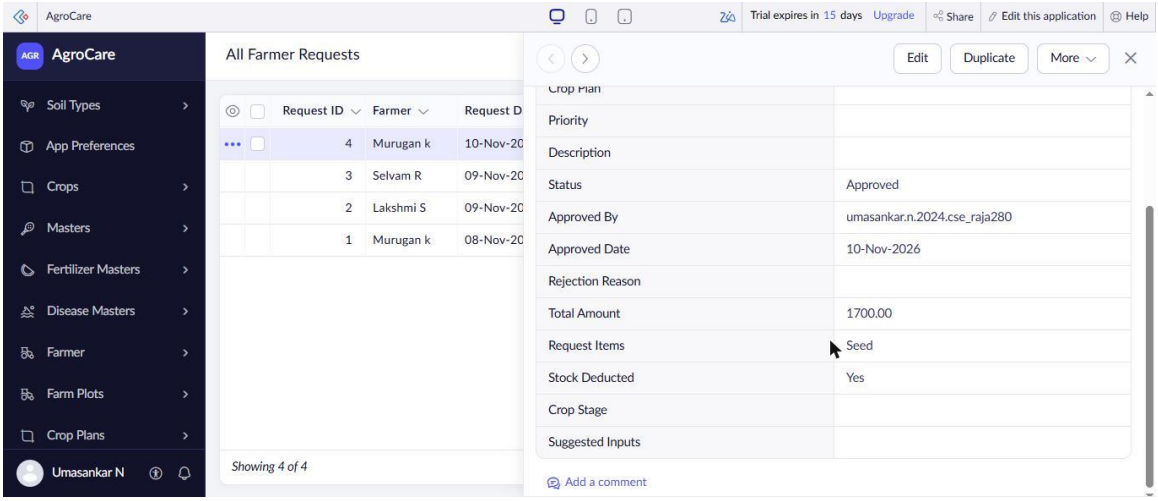


Figure 8.4: Scenario Step 4 – Request Approved and Stock Deducted

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Outcome	Evidence
Normalised data model and forms	21 forms with lookups and 4 subforms	Sections 4.2, 5.2
Automation with Deluge and workflows	15 form workflows and 1 schedule	Sections 5.5, 5.6, Appendix C
Role-based dashboards and reports	12 reports and 3 dashboards	Sections 5.3, 5.4
Integration with Cliq and email	Cliq connection, sendmail notifications	Section 5.7
Access control	Roles and permissions	Section 5.8
Testing and deployment with governance	Nine test cases executed live on 10 November 2026; single development environment (trial)	Chapters 6 and 7

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Aspect	Existing System	AgroCare
Crop selection	Habit and neighbours' advice	Ranked score out of 100 from soil, season, water and pH
Pest/disease handling	Phone call, paper register	Online report, automatic assignment, tracked status
Outbreak detection	After it spreads	Village-level early warning at three similar reports
Reminders	None	Automatic irrigation, fertilizer and stock alerts
Stock and requests	Registers, manual	Automatic costing and deduction
Visibility	None	Dashboards and reports for each role
Access control	None	Role-based permissions

8.4 Limitations of the Low-Code Approach

Several limits were met during development. Formula fields could not be placed in subforms or read other forms, so workflows had to calculate costs. Pivot charts did not list lookup fields, so helper text fields were needed. Some Deluge functions available in documentation were not available in the version used, such as while loops and some collection functions, which needed loop-based workarounds. The roles screen and the permission sets were separate and had to be understood before configuration. The approval and Blueprint features were not available or not needed,

so a status-based flow was used. Dependence on the platform is a vendor lock-in risk, and external integrations such as SMS require third-party providers.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Table 8.3: Placement of AgroCare Features on the Development Spectrum

Level	Features in AgroCare
No-code (configuration)	Forms, lookups, subforms, reports, Kanban boards, pivot charts, role permissions and the Cliq connection
Low-code (Deluge)	Crop score, health score, costing, stock deduction, officer assignment, reminders, early warning and weather advisory
Pro-code (future)	Image-based disease detection, a live weather dashboard widget, and heavy analytics over large data, using Catalyst, widgets or external services

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

AgroCare shows that a complete agricultural advisory and management application can be built on Zoho Creator with forms, workflows, Deluge scripts and dashboards. It helps farmers choose crops with a clear score, record and monitor crop activity, report problems that reach the right officer immediately, and receive warnings and reminders. It gives officers a case board and village-level early warning, and it gives administrators control over stock and requests. Most of the work was done through configuration, with Deluge used for business rules.

The project met its objectives of modelling the data, automating the processes, providing role-based dashboards, integrating notifications and controlling access. The main lessons were the importance of matching link names and field types, and of designing around platform limits.

9.2 Future Enhancements

- Connect a live weather API through invokeUrl and replace manual weather entry.
- Add SMS and WhatsApp notifications for farmers without email.
- Use Zia or an external model to detect diseases from crop photos.
- Move farmers to a customer portal and add local-language support.
- Add market price tracking and yield forecasting.
- Use Zoho Analytics for deeper trend analysis and district-level reports.
- Add the Creator approval process and Blueprint for formal request approval.

9.3 Learning Outcomes and Career Relevance

The project developed skills in data modelling, form and report design, Deluge scripting, workflow design, integration through connections, access control and testing. These skills match the roles of citizen developer, low-code developer, solutions consultant and automation engineer, where quick delivery of business applications and the ability to move to pro-code when needed are valued.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: Nov. 2026).

[3] A. C. Bock and U. Frank, "Low-code platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, Dec. 2021, doi: 10.1007/s12599-021-00726-8.

[4] O. Matvitskyy, A. Jain, K. Davis and A. Leow, "Magic Quadrant for Enterprise Low-Code Application Platforms," Gartner, Inc., Stamford, CT, USA, Jul. 2025.

[5] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika and J. Nisha, "Crop recommendation system for precision agriculture," in *Proc. 8th Int. Conf. Advanced Computing (ICoAC)*, Chennai, India, 2017, pp. 32–36, doi: 10.1109/ICoAC.2017.7951740.

[6] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: Nov. 2026).

[7] Zoho Corporation, "Zoho Cliq Developer Documentation." [Online]. Available: https://www.zoho.com/cliq/help/ (Accessed: Nov. 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section
I	Application created & sections/menu organised	Y	AgroCare app; auto-generated menu	5.1
II	Forms with ≥ 8 distinct field types	Y	Farm_Plot, Inventory, Pest_Disease_Report	5.2
II	Lookup field	Y	Farm_Plot, Crop_Plan, Alert	4.2, 5.2
II	Subform	Y	Fertilizers Applied, Request Items, Action Items, Recommendation Results	4.2, 5.2
II	Field validations / layout rules	Y	pH checks, mandatory fields, duplicate phone	5.2
II	List report with filter, grouping, custom action	Partly	Open_Reports, Low_Stock_Items (no custom action)	5.3
II	Kanban / Calendar / Timeline / Map report	Y (Kanban)	Case_Board, Requests_by_Status	5.3
II	Pivot table / chart report	Y	Cases_by_Category, Farmer_wise_Issues and others	5.3
II	Page / dashboard built with UI Builder	Y	Farmer, Officer and Admin dashboards	5.4
II	Deluge – form events	Y	On validate and on success workflows	5.5
II	Deluge – functions, fetch, loops, collections	Y	Recommend Crops, Daily Checks	5.5
II	Email / in-app notifications	Y	sendmail; Alert records	5.5, 5.6
II	Form workflow	Y	15 workflows	5.6
II	Scheduled workflow	Y	Daily Checks	5.6
II	Approval process	N	Status-based approval used instead	5.6
II	Blueprint	N	Not used	5.6
II	Batch / report workflow	N	Not used	5.6
II	Testing & debugging evidence	Partly	Chapter 6; 9 of 18 test cases executed live	6
III	Integration with a Zoho service	Y	Zoho Cliq	5.7
III	Connection / REST API call (invokeurl)	Y	Cliq connection and invokeUrl	5.7
III	Webhook or Creator REST API	N	Available but not exercised	5.7
III	Users, roles and permission profiles	Partly	Roles CEO, Agricultural Officer, Farmer; Read/Write profiles	5.8
III	Record-level access criteria / customer portal	Partly	Login Email criteria	5.8
III	Zia / AI feature	N	Rule-based scoring only	5.9

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section
III	Analytics (charts / Zoho Analytics)	Y	Pivot charts and dashboards	5.9
IV	Performance optimisation applied & measured	Partly	Criteria, daily schedule	7.2
IV	Deployment across environments	N	Single development environment (trial account)	7.1
IV	Security controls	Partly	OAuth connection, permissions	7.3
IV	Governance (audit trail, administration)	Y	Audit trail, naming convention, backup export	7.4
V	Pro-code extension	N	Justified in 5.10	5.10
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5
—	Mobile app / portal access	N	Not tested in this version	5.11

Total features implemented: 18 of 32 marked Y, 6 marked Partly and 8 not implemented (justified in the sections listed). Items marked Partly or N reflect the live application as it stands.

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome	Evidenced In	Self-Assessed Level
CO1	Principles, architecture and business value of low-code development	Ch. 1, 2	2
CO2	Develop applications using forms, reports, workflows and Deluge	Ch. 4, 5.1–5.6, 6	3
CO3	Integrate with Zoho services and external systems, role-based security	Sec. 4.6, 5.7–5.9	2
CO4	Deploy, secure and manage applications	Ch. 7	2
CO5	Evaluate low-code strategies and the move to pro-code	Sec. 5.9, 5.10, 8.4, 8.5, 9.3	2

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The scripts below are grouped by form and event. Link names are as used in the application.

C.1 Fill Plan Name (DS-01) – Crop_Plan

```
// Script ID : DS-01
// Location  : Crop_Plan -> Create or Edit -> On validate
// Purpose   : Build Plan Name and copy Farmer from the plot
plot = Farm_Plot[ID == input.Plot];
crop = Crop[ID == input.Crop];
input.Plan_Name = plot.Plot_Name + " - " + crop.Crop_Name + " - " +
input.Season;
input.Farmer = plot.Farmer;
```

C.2 Calculate Harvest Date (DS-02) – Crop_Plan

```
// Script ID : DS-02
// Location  : Crop_Plan -> Create or Edit -> On validate
// Purpose   : Expected harvest date = sowing date + crop duration
crop = Crop[ID == input.Crop];
if(crop.Duration_Days != null && input.Sowing_Date != null)
{
    input.Expected_Harvest_Date =
input.Sowing_Date.addDay(crop.Duration_Days);
}
```

C.3 Fill Farmer in Logs (DS-03) – Crop_Health_Log and Irrigation_Log

```
// Script ID : DS-03
// Location  : Crop_Health_Log and Irrigation_Log -> On validate
plan = Crop_Plan[ID == input.Crop_Plan];
if(plan.Farmer != null)
{
    input.Farmer = plan.Farmer;
}
```

C.4 Fertilizer Usage Calculations (DS-04) – Fertilizer_Usage

```
// Script ID : DS-04
// Location  : Fertilizer Usage -> Create or Edit -> On validate
plan = Crop_Plan[ID == input.Crop_Plan];
if(plan.Farmer != null)
{
    input.Farmer = plan.Farmer;
}
total = 0.0;
for each row in input.Fertilizers Applied
{
    fert = Fertilizer_Master[ID == row.Fertilizer];
    price = fert.Price_Per_Unit;
    row.Unit_Price = price;
    row.Cost = row.Quantity * price;
    total = total + (row.Quantity * price);
}
```

```
input.Total Cost = total;
```

C.5 Request Calculations (DS-05) – Farmer_Request

```
// Script ID : DS-05
// Location  : Farmer_Request -> Create or Edit -> On validate
total = 0.0;
for each row in input.Request Items
{
    price = 0.0;
    if(row.Item_Type == "Seed" && row.Seed != null)
    {
        seedRec = Seed_Master[ID == row.Seed];
        price = seedRec.Price_Per_Unit;
    }
    else if(row.Item_Type == "Fertilizer" && row.Fertilizer != null)
    {
        fertRec = Fertilizer_Master[ID == row.Fertilizer];
        price = fertRec.Price_Per_Unit;
    }
    row.Unit_Price = price;
    row.Line_Total = row.Quantity * price;
    total = total + (row.Quantity * price);
}
input.Total Amount = total;
```

C.6 Pest Report Setup (DS-06) – Pest_Disease_Report

```
// Script ID : DS-06
// Location  : Pest_Disease_Report -> Create -> On success
// Purpose   : Assign officer, set status, email officer, post to Cliq
plan = Crop_Plan[ID == input.Crop_Plan];
farmerRec = Farmer[ID == plan.Farmer];
officerList = Officer[Village_Covered == farmerRec.Village];
if(officerList.count() == 0)
{
    officerList = Officer[ID != 0];
}
officerName = "";
officerEmail = "";
for each off in officerList
{
    if(officerName == "")
    {
        officerName = off.Officer_Name;
        officerEmail = off.Email;
    }
}
rep = Pest_Disease_Report[ID == input.ID];
rep.Farmer = plan.Farmer;
rep.Assigned_Officer = officerName;
rep.Status = "Under Review";
if(officerEmail != "")
{
    sendmail
    [
        from :"<verified sender address>"
        to :officerEmail
        subject :"New pest/disease report: " + plan.Plan_Name
        message :"Severity: " + input.Severity + "<br>Symptoms: " +
input.Symptoms
```

```
    ]
}
payload = Map();
payload.put("text", "New " + input.Severity + " pest/disease report: "
+ plan.Plan_Name + ". Village: " + farmerRec.Village);
resp = invokeUrl
[
    url :"https://cliq.zoho.in/api/v2/channelsbyname/agrocare/message"
    type :POST
    parameters :payload.toString()
    connection :"zoho oauth connection"
];
info resp;
```

C.7 Advice Follow Up (DS-07) – Officer_Advice

```
// Script ID : DS-07
// Location  : Officer_Advice -> Create -> On success
farmerRec = Farmer[ID == input.Farmer];
if(input.Pest_Disease_Report != null)
{
    rep = Pest_Disease_Report[ID == input.Pest_Disease_Report];
    rep.Status = "Advisory Given";
}
if(input.Farmer Request != null)
{
    req = Farmer_Request[ID == input.Farmer_Request];
    req.Status = "Issued";
}
insert into Alert
[
    Farmer = input.Farmer
    Alert_Type = "General"
    Severity = "Info"
    Title = "Officer advice received"
    Message = input.Advice
    Alert_Date = zoho.currenttime
]
```

C.8 Approve Request and Deduct Stock (DS-09) – Farmer_Request

```
// Script ID : DS-09
// Location  : Farmer_Request -> Edit -> On success
if(input.Status == "Approved" && input.Stock_Deducted != "Yes")
{
    enough = true;
    for each row in input.Request_Items
    {
        available = 0.0;
        if(row.Item_Type == "Seed" && row.Seed != null)
        {
            for each inv in Inventory[Seed == row.Seed]
            {
                available = available + inv.Current_Stock;
            }
            if(available < row.Quantity) { enough = false; }
        }
        else if(row.Item_Type == "Fertilizer" && row.Fertilizer !=
null)
        {
            for each inv2 in Inventory[Fertilizer == row.Fertilizer]
```

```
            {
                available = available + inv2.Current_Stock;
            }
            if(available < row.Quantity) { enough = false; }
        }
    }
    reqRec = Farmer_Request[ID == input.ID];
    if(enough)
    {
        for each row in input.Request_Items
        {
            if(row.Item_Type == "Seed" && row.Seed != null)
            {
                for each inv3 in Inventory[Seed == row.Seed]
                {
                    inv3.Current_Stock = inv3.Current_Stock -
row.Quantity;
                }
            }
            else if(row.Item_Type == "Fertilizer" && row.Fertilizer !=
null)
            {
                for each inv4 in Inventory[Fertilizer ==
row.Fertilizer]
                {
                    inv4.Current_Stock = inv4.Current_Stock -
row.Quantity;
                }
            }
        }
        reqRec.Stock_Deducted = "Yes";
    }
    else
    {
        reqRec.Status = "Out of Stock";
    }
}
```

C.9 pH Range Check (DS-12) – Soil_Type and Crop

```
// Script ID : DS-12
// Location  : Soil_Type and Crop -> Create or Edit -> On validate
if(input.Min_pH > input.Max_pH)
{
    alert "Min pH cannot be greater than Max pH";
    cancel submit;
}
if(input.Min_pH < 0 || input.Max_pH > 14)
{
    alert "pH must be between 0 and 14";
    cancel submit;
}
```

C.10 Recommend Crops (DS-08)

Location: Crop_Recommendation -> On validate. Condensed from the script in the Application IDE.

```
// Script ID : DS-08
```

```
// Location  : Crop_Recommendation -> On validate
plot = Farm_Plot[ID == input.Plot];
input.Soil_Type = plot.Soil_Type;
input.Soil_PH = plot.Soil_PH;
input.Water_Availability = plot.Water_Availability;
waterLevel = 2;
if(plot.Water_Availability == "Low") { waterLevel = 1; }
if(plot.Water_Availability == "High") { waterLevel = 3; }
names = List();
scores = List();
for each c in Crop[ID != 0]
{
    score = 0;
    if(c.Suitable_Soils.contains(plot.Soil_Type)) { score = score +
35; }
    if(c.Season == input.Season) { score = score + 30; }
    cropLevel = 2;
    if(c.Water_Requirement == "Low") { cropLevel = 1; }
    if(c.Water_Requirement == "High") { cropLevel = 3; }
    waterDiff = abs(cropLevel - waterLevel);
    if(waterDiff == 0) { score = score + 20; }
    else if(waterDiff == 1) { score = score + 10; }
    if(plot.Soil_pH >= c.Min_pH && plot.Soil_pH <= c.Max_pH) { score =
score + 15; }
    else if(plot.Soil_pH >= c.Min_pH - 0.5 && plot.Soil_pH <= c.Max_pH
+ 0.5) { score = score + 7; }
    if(score >= 40) { names.add(c.Crop_Name); scores.add(score); }
}
// repeated best-pick loop sorts crops by score (Deluge has no while
loop)
input.Recommendation_Results.clear();
rank = 1;
summary = "";
used = List();
for each i in scores.indices()
{
    best = -1;
    for each j in scores.indices()
    {
        if(!used.contains(j) && (best == -1 || scores.get(j) >
scores.get(best))) { best = j; }
    }
    if(best != -1)
    {
        used.add(best);
        row = Recommendation_Results();
        row.Crop_Name = names.get(best);
        row.Score = scores.get(best);
        row.Rank = rank;
        input.Recommendation_Results.add(row);
        summary = summary + names.get(best) + " (" + scores.get(best)
+ "/100), ";
        rank = rank + 1;
    }
}
input.Recommended_Crops = summary;
input.Match_Count = names.size();
```

C.11 Weather Advisory (DS-10)

Location: Weather_Update -> Create -> On success. Condensed from the script in the Application IDE.

```
// Script ID : DS-10
// Location  : Weather_Update -> Create -> On success
created = 0;
for each f in Farmer[Village == input.Village && Status == "Active"]
{
    if(input.Rainfall >= 50)
    {
        insert into Alert
        [
            Farmer = f.ID
            Alert Type = "Weather"
            Severity = "Critical"
            Title = "Heavy rain expected"
            Alert_Date = zoho.currenttime
        ]
        created = created + 1;
    }
    for each p in Crop_Plan[Farmer == f.ID && (Status == "Sown" ||
Status == "Growing")]
    {
        for each d in Disease_Master[Crop == p.Crop]
        {
            if(input.Temperature >= d.Min_Temp && input.Temperature <=
d.Max_Temp && input.Humidity >= d.Min_Humidity)
            {
                insert into Alert
                [
                    Farmer = f.ID
                    Alert_Type = "Disease Risk"
                    Severity = "High"
                    Title = d.Disease_Name + " risk for " +
p.Plan_Name
                    Alert Date = zoho.currenttime
                ]
                created = created + 1;
            }
        }
    }
    if(input.Temperature >= 38)
    {
        insert into Alert
        [
            Farmer = f.ID
            Alert_Type = "Weather"
            Severity = "Warning"
            Title = "Hot weather"
            Alert_Date = zoho.currenttime
        ]
        created = created + 1;
    }
}
input.Alerts_Created = created;
```

C.12 Daily Checks (DS-11)

Location: Schedule -> Daily 6:00 AM. Condensed from the script in the Application IDE.

```
// Script ID : DS-11
// Location  : Schedule -> Daily 6:00 AM (Asia/Kolkata); status in the
account: inactive
```

```
// 1. Low stock check
for each inv in Inventory[Current_Stock <= Reorder_Level]
{
    if(Alert[Title == "Low stock: " + inv.Item_Name && Alert_Date >=
zoho.currentdate].count() == 0)
    {
        insert into Alert [ Title = "Low stock: " + inv.Item_Name
Severity = "Warning"  Alert Type = "Stock" ]
    }
}
// 2. Health score for active plans
for each p in Crop_Plan[Status == "Growing"]
{
    score = 0;
    if(Irrigation Log[Crop Plan == p.ID && Irrigation Date >=
zoho.currentdate.subDay(7)].count() > 0) { score = score + 25; }
    if(Fertilizer_Usage[Crop_Plan == p.ID && Usage_Date >=
zoho.currentdate.subDay(30)].count() > 0) { score = score + 20; }
    if(Pest_Disease_Report[Crop_Plan == p.ID && Status !=
"Resolved"].count() == 0) { score = score + 25; }
    if(Alert[Farmer == p.Farmer && Alert_Type == "Disease Risk" &&
Alert_Date >= zoho.currentdate.subDay(7)].count() == 0) { score =
score + 15; }
    if(Crop_Health_Log[Crop_Plan == p.ID && Condition == "Good" &&
Log_Date >= zoho.currentdate.subDay(14)].count() > 0) { score = score
+ 15; }
    p.Health_Score = score;
    if(score >= 85) { p.Health_Rating = "Excellent"; }
    else if(score >= 65) { p.Health_Rating = "Good"; }
    else if(score >= 45) { p.Health_Rating = "Fair"; }
    else { p.Health_Rating = "Poor"; }
}
// 3. Irrigation reminders and 4. fertilizer reminders create Alert
records and send email (same pattern)
// 5. Community early warning
for each v in Farmer.Village.getAll()
{
    reports = Pest_Disease_Report[Village == v && Report_Date >=
zoho.currentdate.subDay(7)];
    // three or more reports of the same disease in one village ->
Critical, High-risk alert and email
}
```

C.13 Fill Inventory Item Name (Workflow 9)

Location: Inventory -> On validate. Condensed from the script in the Application IDE.

```
// Location  : Inventory -> On validate
if(input.Item_Type == "Seed" && input.Seed != null)
{
    input.Item_Name = Seed_Master[ID == input.Seed].Seed_Name;
}
else if(input.Item_Type == "Fertilizer" && input.Fertilizer != null)
{
    input.Item_Name = Fertilizer_Master[ID ==
input.Fertilizer].Fertilizer_Name;
}
```

C.14 Low Stock Alert (Workflow 14)

Location: Inventory -> Create/Edit -> On success. Condensed from the script in the Application IDE.

```
// Location  : Inventory -> Create/Edit -> On success
if(input.Current_Stock <= input.Reorder_Level)
{
    insert into Alert
    [
        Title = "Low stock: " + input.Item Name
        Severity = "Warning"
        Alert Type = "Stock"
        Alert_Date = zoho.currenttime
    ]
    sendmail
    [
        from :zoho.adminuserid
        to :"store@yourorg.com"
        subject :"Low stock: " + input.Item_Name
        message :"Current stock " + input.Current_Stock
    ]
}
```

C.15 Dashboard KPI Cards (DS-13)

Location: Farmer, Officer and Admin dashboard pages. Condensed from the script in the Application IDE.

```
// Location  : Admin_Dashboard page (ZML); counts are computed with
Deluge in the page snippet
farmers = Farmer[Status == "Active"].count();
plots = Farm_Plot[ID != 0].count();
crops = Crop_Plan[ID != 0].count();
pending = Farmer_Request[Status == "Pending"].count();
openCases = Pest Disease Report[Status != "Resolved"].count();
lowStock = Inventory[Current_Stock <= Reorder_Level].count();
// Values shown in the live application: 4 farmers, 5 farm plots, 5
active crops, 0 pending requests, 4 open cases, 2 low stock items
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

- Open the application URL in Chrome, Edge or Firefox, or open the Zoho Creator mobile app.
- Sign in with the email invited by the administrator.
- The left menu shows the forms, reports and dashboards allowed for your role.

D.2 Instructions for the Farmer

- Open Farmer_Dashboard to see active crops, health score, unread alerts and pending requests.
- Add your plots in Farm_Plot with soil type, pH and water availability.
- Open Crop_Recommendation, choose a plot and season, and read the ranked crops.
- Add a Crop_Plan for the chosen crop and record health, irrigation and fertilizer use regularly.
- To report a problem, open Pest_Disease_Report, choose the crop plan, add symptoms, severity and a photo, and submit.
- To request seeds or fertilizer, open Farmer_Request, add items and submit; check Requests_by_Status.
- Read alerts in My_Alerts.

D.3 Instructions for the Agricultural Officer

- Open Officer_Dashboard to see new, under-review and high-risk cases.
- Open Open_Reports or Case_Board and select a case.
- Open Officer_Advice, link the report or request, enter the diagnosis, advice and action items, and submit.
- Update the case to Monitoring or Resolved when appropriate.
- Enter daily weather in Weather_Update to generate advisories for a village.

D.4 Instructions for the Administrator

- Maintain Soil_Type, Crop, Seed_Master, Fertilizer_Master and Disease_Master.
- Maintain Inventory and review Low_Stock_Items.
- Open Farmer_Request and set Status to Approved or Rejected; approval deducts stock.
- Use Admin_Dashboard for an overview, and manage users and roles in Settings.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Umasankar N	Register No.	2116240701574
Project Title	AgroCare – Smart Agriculture Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Umasankar N	Register No.	2116240701574
Department / Section	Computer Science and Engineering	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____